

Níže je připravený studijní materiál k okruhu "Číselné soustavy a převody mezi nimi. Způsoby kódování čísel s pevnou a s pohyblivou řádovou tečkou. Kódování záporných čísel."

9. Číselné soustavy a převody mezi nimi. Způsoby kódování čísel s pevnou a s pohyblivou řádovou tečkou. Kódování záporných čísel.

A. Číselné soustavy a převody

Počítače pracují v pozičních číselných soustavách. Hodnota čísla je dána součtem hodnot jednotlivých cifer vynásobených mocninou základu soustavy podle jejich pozice.

Obecný zápis čísla v poziční soustavě o základu b s $n + 1$ celočíselnými ciframi a m desetinnými ciframi je:
$$(a_n a_{n-1} \dots a_1 a_0 . a_{-1} a_{-2} \dots a_{-m})_b = \sum_{i=-m}^n a_i b^i$$
 kde a_i jsou cifry v soustavě o základu b a b je základ soustavy.

Pro státní závěrečné zkoušky je nutné perfektně ovládat čtyři základní soustavy:

- **Binární (dvojková, $b = 2$)**
 - Používá cifry 0, 1.
 - Základní pro digitální logiku a veškeré digitální systémy.
- **Oktalová (osmičková, $b = 8$)**
 - Používá cifry 0 až 7.

- Dnes méně častá, ale historicky se používala pro kompaktnější zápis binárních čísel (3 bity = 1 oktalová cifra). Příkladem použití jsou přístupová práva v Unixových systémech (např. `chmod 755`).

- **Decimální (desítková, $b = 10$)**

- Používá cifry 0 až 9.
- Přirozená pro lidi, používá se pro vstup a výstup dat.

- **Hexadecimální (šestnáctková, $b = 16$)**

- Používá cifry 0 až 9 a písmena A až F (kde A=10, B=11, ..., F=15).
- Velmi důležitá v IT, protože 1 hexadecimální cifra reprezentuje přesně 4 bity (půl bajtu / nibble). Používá se pro adresy paměti, barvy (RGB), MAC adresy a pro kompaktní zobrazení binárních dat.

Převody mezi soustavami

- **Z libovolné soustavy do desítkové**

- **Princip:** Proveďte se polynomický rozvoj. Cifry se vynásobí příslušnou mocninou základu podle jejich pozice a sečtou se.
- **Příklad:**
 - $(1101)_2 = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 8 + 4 + 0 + 1 = (13)_{10}$
 - $(2A)_{16} = 2 \cdot 16^1 + A \cdot 16^0 = 2 \cdot 16 + 10 \cdot 1 = 32 + 10 = (42)_{10}$

- **Z desítkové soustavy do libovolné**

- **Princip pro celou část:** Celá část čísla se postupně dělí základem nové soustavy a zapisují se zbytky po dělení (odspodu nahoru).
- **Příklad (převod $(13)_{10}$ do binární):**
 - $13 \div 2 = 6$ zbytek 1
 - $6 \div 2 = 3$ zbytek 0
 - $3 \div 2 = 1$ zbytek 1

- $1 \div 2 = 0$ zbytek 1
 - Výsledek: $(1101)_2$ (čteno zdola nahoru)
 - **Princip pro desetinnou část:** Desetinná část se naopak novým základem násobí a sepisují se celé části, které "přetečou" přes řádovou tečku (shora dolů). Proces se opakuje, dokud se desetinná část nestane nulou nebo dokud není dosaženo požadované přesnosti.
 - **Příklad (převod $(0.625)_{10}$ do binární):**
 - $0.625 \cdot 2 = 1.25 \rightarrow 1$ (celá část)
 - $0.25 \cdot 2 = 0.5 \rightarrow 0$ (celá část)
 - $0.5 \cdot 2 = 1.0 \rightarrow 1$ (celá část)
 - Výsledek: $(0.101)_2$
- **Mezi binární a hexadecimální (přímý převod)**
 - **Princip:** Protože $16 = 2^4$, stačí binární číslo rozdělit od řádové tečky (doprava i doleva) do skupin po 4 bitech. Každou čtveřici bitů pak přímo přepíšeme na jednu hexadecimální cifru. Pokud na okrajích nezbyvá dostatek bitů, doplní se nuly.
 - **Příklad:**
 - $(11001011)_2 \rightarrow \underline{1100}_2 \backslash \underline{1011}_2 \rightarrow (CB)_{16}$
 - $(10110.11011)_2 \rightarrow \underline{0001}_2 \backslash \underline{0110}_2 \cdot \underline{1101}_2 \backslash \underline{1000}_2 \rightarrow (16.D8)_{16}$ (doplnění nul na okrajích)

B. Kódování záporných čísel

Pro celá čísla (integers) musíme vyřešit, jak v binární soustavě reprezentovat znaménko mínus, když máme k dispozici jen nuly a jedničky. Používají se tři hlavní přístupy, přičemž v moderních procesorech dominuje ten poslední.

1. Přímý kód (Sign-and-Magnitude)

- **Princip:** Nejvyšší (nejvíce levý) bit se vyhradí čistě pro znaménko (0 = kladné, 1 = záporné). Zbytek bitů nese absolutní hodnotu čísla.
- **Příklad (pro 4 bity):**

- $(+3)_{10} = (0011)_2$
- $(-3)_{10} = (1011)_2$
- **Nevýhody:**
 - **Dvě reprezentace nuly:** Existují dvě nuly: $(0000)_2$ (+0) a $(1000)_2$ (-0), což komplikuje logiku porovnávání v procesoru.
 - **Složitost aritmetiky:** Matematické operace (např. sčítání kladného a záporného čísla) vyžadují odlišné hardwarové obvody než běžné sčítání, nebo složitou logiku pro zpracování znamének.

2. Inverzní kód (Ones' Complement)

- **Princip:** Záporné číslo vznikne prostým znegováním (překlopením) všech bitů kladného čísla (z 0 na 1 a naopak).
- **Příklad (pro 4 bity, vyjádření -3):**
 - Kladná 3 je $(0011)_2$.
 - Inverze všech bitů: $(1100)_2$ (výsledná -3 v inverzním kódu).
- **Nevýhody:**
 - **Dvě reprezentace nuly:** Stále trpí problémem dvou nul: $(0000)_2$ (+0) a $(1111)_2$ (-0).
 - **Složitější aritmetika:** Sčítání vyžaduje speciální úpravu pro přenos z nejvyššího bitu (tzv. "end-around carry").

3. Doplnkový kód (Two's Complement) – Standard v moderních CPU

- **Princip:** Záporné číslo vytvoříme tak, že vezmeme jeho kladnou hodnotu, provedeme bitovou inverzi (překlopíme všechny bity) a k výsledku přičteme jedničku.
- **Příklad (pro 4 bity, vyjádření -3):**
 1. Kladná 3 je $(0011)_2$.
 2. Inverze všech bitů (jedničkový doplněk): $(1100)_2$.
 3. Přičtení jedničky: $(1100)_2 + (0001)_2 = (1101)_2$ (výsledná -3 v doplňkovém kódu).
- **Výhody:**

- **Jedna unikátní nula:** Pouze $(0000)_2$ reprezentuje nulu. Když zneguješ nulu (samé jedničky) a přičteš 1, dostaneš $(0000)_2$ (přetečení do vyššího bitu se ignoruje).
- **Jednotný hardware pro aritmetiku:** Sčítání a odčítání funguje naprosto stejně pro kladná i záporná čísla. Procesor nepotřebuje speciální obvod pro odčítání; operaci $A - B$ provede jako $A + (-B)$, kde $(-B)$ je dvojkový doplněk čísla B .
- **Rozsah pro n bitů:** $\langle -2^{n-1}, 2^{n-1} - 1 \rangle$.
 - Například pro 8 bitů je rozsah $\langle -2^{8-1}, 2^{8-1} - 1 \rangle = \langle -128, 127 \rangle$.
 - Nejvyšší bit má zápornou váhu (např. pro 8 bitů je to -2^7).

C. Čísla s pevnou řádovou tečkou (Fixed-point)

- **Princip:** Pozice řádové tečky v binárním slově je pevně daná a neměnná. Určitý počet bitů reprezentuje celou část a zbývající bity reprezentují desetinnou část.
- **Formát:** Obvykle se značí jako $QI.F$, kde I je počet bitů pro celou část a F je počet bitů pro desetinnou část. Celkový počet bitů je $I + F$.
- **Příklad:** Formát $Q8.8$ znamená, že z 16 bitů je 8 vyhrazeno pro celou část a 8 pro desetinnou.
 - Váhy bitů před tečkou jsou $2^7, 2^6, \dots, 2^0$.
 - Váhy bitů za tečkou jsou $2^{-1}(0.5), 2^{-2}(0.25), 2^{-3}(0.125), \dots, 2^{-8}$.
 - Číslo $(00000001.10000000)_{Q8.8}$ by reprezentovalo $1 \cdot 2^0 + 1 \cdot 2^{-1} = 1 + 0.5 = (1.5)_{10}$.
- **Výhody:**
 - **Rychlé výpočty:** Výpočty jsou extrémně rychlé, protože se interně provádějí pomocí standardních hardwarových instrukcí pro celá čísla (Integer ALU).
 - **Přesnost:** Pro daný rozsah je přesnost konstantní.
- **Nevýhody:**
 - **Omezený dynamický rozsah:** Nelze současně reprezentovat extrémně velká a extrémně malá čísla. Pokud výsledek operace

překročí rozsah, dochází k rychlému přetečení nebo ztrátě přesnosti.

- **Nutnost škálování:** Programátor musí ručně spravovat pozici řádové tečky a škálování, aby se zabránilo přetečení nebo podtečení.
- **Použití:** Mikrokontroléry bez FPU (Floating-Point Unit), digitální signálové procesory (DSP), finanční matematika (kde se fixuje počet desetinných míst, aby nevznikaly chyby zaokrouhlením), embedded systémy.

D. Čísla s pohyblivou řádovou tečkou (Floating-point)

Pro reprezentaci reálných čísel s obrovským rozsahem se používá vědecký (semilogaritmický) zápis, standardizovaný normou IEEE 754.

Číslo je v paměti rozděleno do tří základních částí:

- **Znaménko (s , Sign):** 1 bit (0 pro kladné, 1 pro záporné).
- **Exponent (e):** Určuje řádovou velikost čísla (posun tečky). Kódován je v tzv. kódu s posunutou nulou (Biased exponent), aby se nemuselo řešit znaménko exponentu. Hodnota bias je konstanta, která se od uloženého exponentu odečítá, aby se získal skutečný exponent.
- **Mantisa (m , Significand / Fraction):** Reprezentuje samotné číslice. Číslo je vždy normalizováno, což znamená, že před řádovou tečkou je vždy právě jedna jednička ($1.mmm$). Tato jednička se do paměti fyzicky neukládá (je "skrytá"), čímž se ušetří 1 bit přesnosti.

Celková hodnota se spočítá jako:

$$\text{Hodnota} = (-1)^s \cdot (1 + m) \cdot 2^{e - \text{bias}}$$

kde s je hodnota bitu znaménka, m je hodnota mantisy (zlomková část za skrytou jedničkou), e je hodnota uloženého exponentu a bias je posunutí exponentu.

Standardní formáty IEEE 754

- **Single Precision (float v C/C++)**

- Celkem 32 bitů.
- 1 bit znaménko, 8 bitů exponent (bias = 127), 23 bitů mantisa.
- Rozsah: přibližně $\pm 1.18 \times 10^{-38}$ až $\pm 3.4 \times 10^{38}$.
- Přesnost: přibližně 7 desetinných míst.

- **Double Precision (double v C/C++)**

- Celkem 64 bitů.
- 1 bit znaménko, 11 bitů exponent (bias = 1023), 52 bitů mantisa.
- Rozsah: přibližně $\pm 2.23 \times 10^{-308}$ až $\pm 1.80 \times 10^{308}$.
- Přesnost: přibližně 15-17 desetinných míst.

Speciální hodnoty IEEE 754

Norma definuje chování pro extrémní stavy pomocí speciálních konfigurací exponentu a mantisy:

- **Nula:** Kladná i záporná nula (exponent i mantisa jsou samé nuly).
- **Nekonečno ($\pm\infty$):** Exponent má samé jedničky, mantisa má samé nuly. Vznikne například dělením nenulového čísla nulou.
- **NaN (Not a Number):** Exponent má samé jedničky, mantisa je nenulová. Reprezentuje neplatné operace (např. odmocnina ze záporného čísla, dělení nuly nulou 0/0).

Typické zkouškové otázky

1. Převeďte číslo $(25)_{10}$

do dvojkové a šestnáctkové soustavy. Dále převeďte $(1A)_{16}$ do dvojkové a desítkové soustavy.

- **Odpověď:**
 - $(25)_{10}$ do dvojkové:
 - $25 \div 2 = 12$ zbytek 1
 - $12 \div 2 = 6$ zbytek 0
 - $6 \div 2 = 3$ zbytek 0

- $3 \div 2 = 1$ zbytek 1
- $1 \div 2 = 0$ zbytek 1
- Výsledek: $(11001)_2$
- $(25)_{10}$ **do šestnáctkové:**
 - $25 \div 16 = 1$ zbytek 9
 - $1 \div 16 = 0$ zbytek 1
 - Výsledek: $(19)_{16}$
- $(1A)_{16}$ **do dvojkové:**
 - $1_{16} = 0001_2$
 - $A_{16} = 1010_2$
 - Výsledek: $(00011010)_2 = (11010)_2$
- $(1A)_{16}$ **do desítkové:**
 - $\$(1A)_{16} = 1 \cdot 16^1 + A \cdot 16^0 = 1 \cdot 16 + 10 \cdot 1 = 16 + 10 = (26)_{10}\$$

2. Jak funguje dvojkový doplněk a jaké jsou jeho hlavní výhody?

- **Odpověď:** Dvojkový doplněk je metoda kódování záporných celých čísel v binární soustavě. Záporné číslo se získá tak, že se vezme jeho kladná absolutní hodnota, provedou se bitové inverze (0 se změní na 1, 1 na 0) a k výsledku se přičte jednička.
- **Hlavní výhody:**
 - **Jedna unikátní reprezentace nuly:** Na rozdíl od přímého a inverzního kódu existuje pouze jedna nula $(0000)_2$.
 - **Zjednodušená aritmetika:** Sčítání a odčítání funguje pro kladná i záporná čísla stejným hardwarovým obvodem. Odčítání $A - B$ se provede jako sčítání $A + (-B)$, kde $(-B)$ je dvojkový doplněk B . To výrazně zjednodušuje návrh procesorů.
 - **Větší rozsah pro záporná čísla:** Pro n bitů je rozsah $\langle -2^{n-1}, 2^{n-1} - 1 \rangle$, což znamená, že lze reprezentovat o jedno záporné číslo více než kladných (např. pro 8 bitů -128 až $+127$).

3. Co je bit, nibble, byte a word?

- **Odpověď:**

- **Bit (Binary Digit):** Základní jednotka informace v digitálních systémech, která může nabývat pouze dvou stavů: 0 nebo 1.
 - **Nibble (půlbajt):** Skupina 4 bitů. Jedna hexadecimální cifra přesně odpovídá jednomu nibble.
 - **Byte (bajt):** Skupina 8 bitů. Je to nejmenší adresovatelná jednotka paměti ve většině počítačových architektur a základní jednotka pro ukládání znaků (např. ASCII).
 - **Word (slovo):** Skupina bitů, kterou procesor zpracovává jako jednu jednotku. Velikost slova se liší podle architektury procesoru (např. 16 bitů, 32 bitů, 64 bitů).
-
-

Níže naleznete dokonalý studijní materiál k okruhu "10. Programátorský model procesoru", doplněný a naformátovaný dle vašich přísných pravidel.

10. Programátorský model procesoru

Klíčové pojmy

- instrukce
- registr
- ALU
- program counter (PC)
- stack pointer (SP)
- paměť
- přerušení
- instrukční soubor
- symbolická adresa
- fetch-decode-execute cyklus
- adresní režimy
- konvence volání
- zásobníkový rámec
- návratová adresa
- obslužná rutina přerušení (ISR)
- kontext procesu

10.1 Programátorský model procesoru a registry

Programátorský model procesoru je abstrakce hardwaru, kterou programátor v nízkoúrovňovém jazyce (jako je Assembler) vidí a může ji

přímo ovlivňovat. Zahrnuje především vnitřní paměť procesoru – **registry**, **instrukční sadu**, **adresní režimy** a **viditelný stav procesoru**.

Procesor vykonává instrukce v opakovaném **fetch-decode-execute cyklu**:

1. **Fetch (načtení)**: Procesor načte instrukci z paměti na adrese uložené v Program Counteru (PC). 2. **Decode (dekódování)**: Instrukce je dekována, aby procesor zjistil, jakou operaci má provést a s jakými operandy. 3. **Execute (provedení)**: Procesor provede operaci, například aritmetickou operaci v ALU, přesun dat nebo skok.

Registry jsou malé, velmi rychlé paměti přímo v procesoru, které slouží k ukládání dat, adres a řídicích informací. Dělíme je podle jejich účelu:

- **Všeobecné registry (GPR - General Purpose Registers):**
 - Slouží k rychlému ukládání operandů a mezivýsledků aritmetických a logických operací.
 - Příklady v architektuře x86/x64: EAX / RAX (akumulátor), EBX / RBX (báze), ECX / RCX (čítač pro cykly), EDX / RDX (data).
- **Ukazatel instrukcí (IP - Instruction Pointer / PC - Program Counter):**
 - Obsahuje adresu paměťového místa, kde se nachází následující instrukce, která se má vykonat.
 - Procesor jej automaticky inkrementuje po načtení instrukce.
- **Ukazatel zásobníku (SP - Stack Pointer):**
 - Ukazuje na vrchol zásobníku (Stack) v operační paměti RAM.
 - Zásobník slouží k ukládání návratových adres z podprogramů, lokálních proměnných a parametrů funkcí.
- **Stavový registr (Flags / Status Register):**
 - Obsahuje jednotlivé bity (příznaky), které reflektují výsledek poslední provedené operace (např. v **ALU - Aritmeticko-logické jednotce**).
 - Klíčové příznaky:

- ZF (Zero Flag) – nastaven, pokud výsledek operace byl nula.
- CF (Carry Flag) – nastaven, pokud došlo k přenosu z nejvyššího řádu (přetečení bezznaménkového čísla).
- OF (Overflow Flag) – nastaven, pokud došlo k aritmetickému přetečení (u čísel se znaménkem).
- SF (Sign Flag) – nastaven, pokud je výsledek záporný.

10.2 Instrukce a instrukční soubor (ISA)

Instrukční soubor (Instruction Set Architecture - ISA) je sada všech instrukcí, kterým daný procesor rozumí (např. x86, ARM, RISC-V). Definuje, jaké operace procesor umí.

Strojová instrukce je binární slovo, které se skládá ze dvou hlavních částí: 1.

Operační znak (Opcode): Identifikuje, o jakou operaci se jedná (např. sčítání, přesun dat, skok). 2. **Operandy:** Specifikují data, se kterými se má operace provést. Mohou to být registry, přímé hodnoty (literály) nebo adresy v paměti.

Typy operací:

- **Aritmetické a logické operace:** Prováděné v ALU, např. sčítání (ADD), odečítání (SUB), bitové operace (AND , OR , XOR).
- **Přesuny dat:** Přesun dat mezi registry, mezi registrem a pamětí, nebo načtení konstanty. Příklad: MOV EAX, EBX (přesune obsah EBX do EAX).
- **Skoky a volání podprogramů:** Mění tok řízení programu. Příklad: JMP , CALL .
- **I/O operace:** Slouží ke komunikaci s perifériemi (klávesnice, displej, síťová karta).
 - Bud' přes speciální instrukce (např. IN , OUT v x86).
 - Nebo mapováním periférií přímo do paměťového prostoru (Memory-Mapped I/O), kde se s nimi pracuje jako s běžnou pamětí.

10.3 Jazyk symbolických adres (JSA / Assembler)

Strojový kód (posloupnost binárních nul a jedniček) je pro lidi nečitelný.

Jazyk symbolických adres (Assembler) ho nahrazuje textovými zkratkami (mnemoniky), které jsou pro programátora srozumitelnější.

Symbolická adresa (Návěští / Label):

- Namísto ručního vypočítávání absolutních paměťových adres (např. `0x004F3A`) v definici dat nebo při skocích používáme textové štítky (např. `loop_start:`, `my_variable:`).
- Překladač (Assembler) pak při generování strojového kódu tyto symbolické adresy automaticky nahradí reálnými číselnými adresami.

Sekvence instrukcí a algoritmizace základních úloh

V JSA neexistují přímo struktury jako `if-else`, `while` nebo `for`. Vše se staví pomocí instrukcí porovnání (např. `CMP`) a podmíněných nebo nepodmíněných skoků.

Příklad realizace `if (A == B)`:

```
CMP EAX, EBX      ; Porovná hodnoty v registrech EAX a
EBX
JNE rovno_ne      ; Jump if Not Equal: Pokud se
nerovnají, skočí na návěští 'rovno_ne'
; --- Kód, který se provede, pokud A == B ---
MOV ECX, 1        ; Příklad: Nastaví ECX na 1
JMP konec_if      ; Nepodmíněný skok přes větev "else"

rovno_ne:
; --- Kód pro větev else ---
MOV ECX, 0        ; Příklad: Nastaví ECX na 0
```

```
konec_if:
; --- Pokračování programu ---
```

Podobně se realizují i cykly, kde se na konci těla cyklu provede podmíněný skok zpět na začátek cyklu, dokud není splněna ukončovací podmínka.

10.4 Časování programu, podprogramy a přerušení

Časování programu

- Každá instrukce potřebuje k provedení určitý počet taktů procesoru (clock cycles).
- Instrukce pracující jen s registry (např. `ADD`) trvají obvykle 1 takt, zatímco instrukce přistupující do paměti (např. `MOV` z RAM) nebo složitější aritmetické operace (např. dělení) mohou trvat násobně déle.
- U jednoduchých architektur lze časování programu přesně spočítat. U moderních CPU s pipeline, cache a predikcí větvení se doba provedení instrukce odhaduje hůře kvůli složitosti a dynamickému chování.
- **Důležité:** Přístup do paměti bývá výrazně dražší než operace v registrech.

Podprogramy (Funkce)

Volání podprogramů se opírá o **programový zásobník (Stack)**.

- **Volání podprogramu (`CALL`):**
 - Při instrukci volání (`CALL podprogram`) procesor vezme aktuální hodnotu z ukazatele instrukcí (`IP / PC`) – což je **návratová adresa** – a uloží ji na vrchol zásobníku.
 - Poté změní `IP` na adresu začátku podprogramu.
- **Návrat z podprogramu (`RET`):**
 - Na konci podprogramu se zavolá instrukce návratu (`RET`).
 - Ta vyzvedne (popne) návratovou adresu ze zásobníku zpět do `IP` a procesor pokračuje hned za místem, odkud byl

podprogram zavolán.

- **Zásobníkový rámec:** Každé volání podprogramu vytvoří na zásobníku tzv. zásobníkový rámec, který obsahuje parametry, lokální proměnné a návratovou adresu.
- **Konvence volání:** Určují, jak se předávají parametry (v registrech nebo na zásobníku) a kdo je zodpovědný za úklid zásobníku po návratu (volající nebo volaný podprogram).

Přerušení (Interrupt)

Přerušení je mechanismus, kterým dokáže hardware nebo software asynchronně přerušit běžící program a vynutit si okamžitou pozornost procesoru.

- **Hardwarové přerušení:**
 - Vyvoláno vnější periferií (např. stisk klávesy, příchod paketu na síťovou kartu, tik hardwarových hodin, chyba paměti).
- **Softwarové přerušení (Exception/Trap):**
 - Vyvoláno záměrně instrukcí v programu (např. pro vyvolání služeb operačního systému – **Syscall**).
 - Nebo chybou v programu (dělení nulou, neplatný přístup do paměti).

Průběh zpracování přerušení: 1. **Dokončení instrukce:** Procesor dokončí aktuálně běžící instrukci. 2. **Uložení kontextu:** Procesor automaticky uloží na zásobník **stavový registr** (Flags) a aktuální hodnotu **ukazatele instrukcí** (IP / PC), aby se mohl k rozdělané práci vrátit. (Některé architektury ukládají i další registry, nebo je ukládá až obslužná rutina). 3. **Skok na ISR:** Procesor se podívá do **tabulky vektorů přerušení (Interrupt Vector Table)** na adresu odpovídající danému číslu přerušení a skočí na příslušnou **obslužnou rutinu (ISR - Interrupt Service Routine)**. 4. **Obsluha události:** ISR obslouží událost (např. přečte znak z klávesnice, zpracuje síťový paket). 5. **Návrat:** ISR je ukončena speciální instrukcí (např. IRET nebo RETI), která obnoví registry ze zásobníku a procesor pokračuje v původním programu, jako by se nic nestalo.

Praktické použití

- **Optimalizace nízkourovňového kódu:** Přímá kontrola nad registry a pamětí umožňuje psát vysoce optimalizovaný kód.
- **Embedded systémy:** V systémech s omezenými zdroji je často nutné programovat přímo v Assembleru pro maximální efektivitu.
- **Ladění chyb paměti:** Pochopení programátorského modelu je klíčové pro analýzu pádů programů a chyb spojených s pamětí.
- **Pochopení operačních systémů:** OS intenzivně využívají registry, zásobník a přerušení pro správu procesů, paměti a I/O.
- **Vývoj ovladačů:** Ovladače hardwaru komunikují přímo s perifériemi, často pomocí I/O operací a přerušení.

Nejčastější chyby u zkoušky

- **Zaměňování registru a paměťové adresy:** Registry jsou přímo v CPU, paměť je externí RAM.
- **Představa, že přerušení je totéž co funkční volání:** Přerušení je asynchronní, vynucené a ukládá více kontextu než běžné volání funkce.
- **Ignorování uloženého kontextu procesu:** Zapomenutí, že při přerušení nebo přepnutí kontextu je nutné uložit stav procesoru.
- **Mechanické použití vzorců bez kontroly definičního oboru:** (Týká se spíše matematiky, ale v kontextu Assembleru by to mohlo být např. ignorování přetečení).

Typické zkouškové otázky

1. Popište fetch-decode-execute cyklus.

- **Odpověď:** Fetch-decode-execute cyklus je základní operační cyklus procesoru. Skládá se ze tří fází:
 1. **Fetch (načtení):** Procesor načte instrukci z paměti na adrese, na kterou ukazuje Program Counter (PC).
 2. **Decode (dekódování):** Načtená instrukce je dekována, aby procesor zjistil, jakou operaci má provést a s jakými

operandy.

3. **Execute (provedení):** Procesor provede dekódovanou operaci, například aritmetickou operaci, přesun dat nebo skok. Po dokončení se PC aktualizuje na adresu další instrukce.

2. Jakou roli mají registry PC a SP?

- **Odpověď:**

- **Program Counter (PC) / Instruction Pointer (IP):**

Obsahuje paměťovou adresu následující instrukce, která má být provedena. Je klíčový pro řízení toku programu, protože procesor jej automaticky inkrementuje po načtení instrukce a mění jej při skocích nebo volání podprogramů.

- **Stack Pointer (SP):** Ukazuje na aktuální vrchol

programového zásobníku v operační paměti. Zásobník se používá pro ukládání návratových adres z podprogramů, lokálních proměnných a parametrů funkcí. SP se automaticky upravuje při operacích **PUSH** (uložení na zásobník) a **POP** (vyzvednutí ze zásobníku).

3. Co se děje při přerušení?

- **Odpověď:** Při přerušení se děje následující sekvence událostí:

1. Procesor dokončí aktuálně prováděnou instrukci.
2. Automaticky uloží na zásobník klíčové části **kontextu procesu**, typicky obsah Program Counteru (návratovou adresu) a stavového registru (Flags).
3. Procesor vyhledá v **tabulce vektorů přerušení** adresu **obslužné rutiny přerušení (ISR)**, která odpovídá danému typu přerušení.
4. Skočí na začátek této ISR a začne ji vykonávat, čímž obslouží událost, která přerušení vyvolala.
5. Po dokončení ISR se provede speciální instrukce (např. **IRET**), která obnoví uložený kontext (PC a Flags) ze zásobníku, a procesor pokračuje v původním programu od místa, kde byl přerušen.

Shrnutí kapitoly

Programátorský model procesoru vysvětluje, jak instrukce pracují s registry, pamětí a tokem řízení. Je to základní abstrakce, kterou programátor vidí a ovládá v nízkoúrovňových jazycích. Pochopení fetch-decode-execute cyklu, rolí registrů (PC, SP, Flags), struktury instrukčního souboru a mechanismů podprogramů a přerušení je klíčové pro efektivní programování, ladění a porozumění fungování operačních systémů a hardwaru.

Následující studijní materiál byl vytvořen na základě studentova textu a doplněn dle požadavků pro státní závěrečné zkoušky.

11. Jazyk C: Základní datové typy a strukturovaný datový typ. Pole a ukazatele, dynamická alokace paměti.

Klíčové pojmy

- Datový typ
- Celočíselný typ (`char` , `short` , `int` , `long` , `long long` , `signed` , `unsigned`)
- Desetinný typ (`float` , `double` , `long double`)
- Logický typ (`_Bool` , `bool`)
- Typ `void`
- Výčtový typ (`enum`)
- Struktura (`struct`)
- Union (`union`)
- Pole (array)
- Ukazatel (pointer)
- Adresa (`&` operátor)
- Dereference (`*` operátor)
- Ukazatelová aritmetika
- Dynamická paměť
- Zásobník (Stack)
- Halda (Heap)
- `malloc`

- `calloc`
- `realloc`
- `free`
- Memory leak
- Dangling pointer
- `NULL` pointer
- Padding
- Typová konverze (casting)

A. Základní a strukturované datové typy

V jazyce C reprezentují datové typy velikost a způsob interpretace dat uložených v paměti. Jsou klíčové pro efektivní využití hardwaru a správnou manipulaci s daty.

1. Základní (skalární) datové typy

Tyto typy uchovávají jednu hodnotu a jsou základními stavebními kameny pro složitější datové struktury.

- **Celočíselné typy:**
 - `char` : Obvykle 1 bajt (8 bitů). Slouží pro reprezentaci znaků (např. ASCII) nebo malých celých čísel. Může být `signed char` (rozsah typicky od -128 do 127) nebo `unsigned char` (rozsah typicky od 0 do 255). Standard C garantuje, že `char` je dostatečně velký pro uložení jakéhokoli znaku z *základní znakové sady*.
 - `short int` (zkráceně `short`) : Celočíselný typ, garantuje minimálně 16 bitů.
 - `int` : Základní celočíselný typ. Jeho velikost je závislá na architektuře systému (dnes standardně 4 bajty / 32 bitů na většině systémů, ale standard C garantuje pouze minimálně 16 bitů).
 - `long int` (zkráceně `long`) : Celočíselný typ, garantuje minimálně 32 bitů.

- `long long int` (zkráceně `long long`): Zaveden ve standardu C99, garantuje minimálně 64 bitů.
- **Modifikátory:**
 - `signed`: Určuje, že číslo může být kladné i záporné (výchozí pro `int`, `short`, `long`, `long long`). Záporná čísla jsou obvykle reprezentována v doplňkovém kódu.
 - `unsigned`: Určuje, že číslo je vždy nezáporné. Tím se posune rozsah kladných hodnot (např. `unsigned char` 0-255 místo -128-127).
- **Desetinné typy (s pohyblivou řádovou čárkou dle IEEE 754):**
 - `float`: Jednoduchá přesnost (obvykle 4 bajty / 32 bitů). Poskytuje přibližně 7 desetinných míst přesnosti.
 - `double`: Dvojitá přesnost (obvykle 8 bajtů / 64 bitů). Poskytuje přibližně 15-17 desetinných míst přesnosti.
 - `long double`: Rozšířená přesnost (velikost závisí na implementaci, může být 10, 12 nebo 16 bajtů).
- **Logický typ:**
 - V původním C (C89) samostatný logický typ neexistoval. Logická hodnota byla reprezentována celým číslem: `0` je nepravda (`false`), jakákoli nenulová hodnota je pravda (`true`).
 - Od standardu C99 existuje typ `_Bool`. Po vložení hlavičkového souboru `<stdbool.h>` je k dispozici makro `bool`, které se mapuje na `_Bool`, a makra `true` a `false`.
- **Typ void:**
 - `void` je neúplný typ, který nemá žádnou hodnotu. Používá se pro:
 - Funkce, které nevrací žádnou hodnotu (např. `void f()`).
 - Funkce, které nepřijímají žádné parametry (např. `int f(void)`).
 - Generické ukazatele (`void*`), které mohou ukazovat na jakýkoli datový typ.
- **Výčtový typ (enum):**
 - Umožňuje definovat sadu pojmenovaných celočíselných konstant. Zvyšuje čitelnost kódu. c `enum Barva { CERVENA, ZELENA, MODRA }; enum Barva mojeBarva = ZELENA;`

2. Strukturované datové typy

Umožňují seskupit více proměnných, potenciálně různých typů, pod jedno jméno.

- **Struktura (struct):**

- Umožňuje seskupit několik spolu souvisejících proměnných (členů), i různých typů, pod jedno společné jméno. Vytváří uživatelský datový typ.
- Celková velikost struktury v paměti je dána součtem velikostí jejích členů, plus případný *padding* (zarovnání), který kompilátor přidává pro optimalizaci přístupu k paměti. ``c struct Student { int id; char jmeno[50]; float prumer; }; // Celková velikost v paměti je dána součtem velikostí členů (+ případný padding pro zarovnání)

```
// Deklarace proměnné typu struct Student struct Student s1; s1.id = 1;
strcpy(s1.jmeno, "Jan Novak"); s1.prumer = 1.5f; * **`typedef` pro
struktury:** Pro zjednodušení zápisu se často používá
`typedef` k vytvoření aliasu pro typ struktury: c typedef
struct Student { int id; char jmeno[50]; float prumer; } Student; // Nyní
lze použít "Student" místo "struct Student"
```

```
Student s2; s2.id = 2; * **Union (`union`):**
```

- Podobně jako `struct` seskupuje členy, ale všechny členy sdílejí stejné paměťové místo. Velikost `union` je rovna velikosti největšího člena. V daný okamžik může `union` uchovávat hodnotu pouze jednoho ze svých členů. c union Data { int i; float f; char s[20]; };

```
union Data data; data.i = 10; // Nyní data.i obsahuje 10 // data.f a
data.s jsou nyní neplatné ``
```

B. Pole (array) a Ukazatele (pointer)

V jazyce C existuje extrémně silná a hluboká vazba mezi poli a ukazateli, často označovaná jako "array-pointer duality".

1. Pole

Homogenní datová struktura tvořená pevným počtem prvků stejného typu, které jsou v paměti uloženy souvisle lineárně za sebou.

- **Deklarace:** `c int pole[5];` // Alokuje v paměti prostor pro 5 celých čísel
- **Indexování:** Prvky pole jsou indexovány od 0 do N-1, kde N je velikost pole. `c pole[0] = 10; pole[4] = 20;`
- **Jméno pole:** Jméno pole (bez indexu) se ve většině výrazů chová jako konstantní ukazatel na jeho první prvek (`&pole[0]`). Nelze mu přiřadit novou adresu.
- **Vícerozměrná pole:** Jsou to pole polí. Například `int matice[3][4];` je pole 3 prvků, kde každý prvek je pole 4 celých čísel. Uložení v paměti je stále souvislé, po řádcích.
- **Pole jako parametry funkcí:** Když je pole předáno funkci, "rozpadne se" (decays) na ukazatel na svůj první prvek. Funkce tak nezná skutečnou velikost pole a je nutné ji předat jako samostatný parametr.

2. Ukazatele (Pointery)

Ukazatel je proměnná, jejíž hodnotou je paměťová adresa jiné proměnné nebo paměťového bloku.

- **Deklarace:** `c int *p;` // Deklaruje 'p' jako ukazatel na int
- **Operátory:**
 - **Referenční operátor (&):** Získá adresu proměnné (operátor "adresa z"). `c int x = 10; int *p = &x;` // Do ukazatele 'p' uložíme adresu proměnné 'x'
 - **Dereferenční operátor (*):** Přistoupí k hodnotě uložené na adrese, kam ukazatel ukazuje. `c *p = 20;` // Dereference: na adresu v 'p' zapíšeme 20, čímž se změní hodnota 'x' na 20

- **NULL ukazatel:** Speciální hodnota, která indikuje, že ukazatel neukazuje na žádnou platnou paměťovou adresu. Je definován v hlavičkových souborech jako `<stddef.h>` nebo `<stdlib.h>`.
c `int *ptr = NULL;`
- **void* (generický ukazatel):** Ukazatel na `void` může uchovávat adresu jakéhokoli datového typu. Pro dereferenci nebo ukazatelovou aritmetiku je nutná explicitní typová konverze (casting).
c `int i = 5; void *gen_ptr = &i; // int val = *gen_ptr; // Chyba: nelze dereferencovat void*
int val = *(int*)gen_ptr; // Správně s castingem`
- **Ukazatelová aritmetika a vztah k polím:**
 - Pokud máme ukazatel na prvek v poli, přičtením jedničky (`p + 1`) neposuneme adresu o jeden bajt, ale o velikost datového typu, na který ukazatel ukazuje (`sizeof(*p)`).
 - Například, pokud `p` ukazuje na `int`, `p + 1` posune ukazatel o `sizeof(int)` bajtů.
 - Zápis `pole[i]` je interně kompilátorem přeložen přesně jako `*(pole + i)`. To demonstruje silnou dualitu mezi poli a ukazateli.
 - Odečítání dvou ukazatelů na prvky stejného pole vrátí počet prvků mezi nimi.

C. Dynamická alokace paměti

Při běžné deklaraci proměnných (např. `int x;`) nebo statických polí (např. `int pole[100];`) se paměť alokuje na **zásobníku (Stack)** nebo v datovém segmentu (pro globální/statické proměnné). Velikost musí být známá v době překladu a po opuštění bloku/funkce paměť automaticky zaniká.

Pokud potřebujeme velikost paměti určit až za běhu programu (runtime) podle vstupu od uživatele nebo dynamických potřeb, musíme použít dynamickou alokaci na **haldě (Heap)**. K tomu slouží funkce z knihovny `<stdlib.h>`:

- `void* malloc(size_t velikost_v_bajtech) :`
 - Alokuje souvislý blok paměti požadované velikosti v bajtech.
 - Vrátí univerzální ukazatel `void*` na začátek tohoto bloku.
 - Paměť neinicizuje (obsahuje náhodná data, která v ní zbyla).
 - V případě selhání alokace (např. nedostatek paměti) vrátí `NULL` .
`c int *dynamicke_pole = (int*) malloc(10 * sizeof(int)); // Explicitní přetypování (int*) je v C volitelné, ale dobrá praxe`
- `void* calloc(size_t pocet_prvku, size_t velikost_prvku) :`
 - Podobně jako `malloc` , ale alokuje prostor pro pole `pocet_prvku` prvků, každý o velikosti `velikost_prvku` bajtů.
 - Celý alokovaný prostor **vynuluje** (zapiše samé nuly).
 - V případě selhání vrátí `NULL` . `c int *dynamicke_pole_nulovane = (int*) calloc(10, sizeof(int));`
- `void* realloc(void* ukazatel, size_t nova_velikost) :`
 - Změní velikost dříve alokovaného bloku paměti, na který ukazuje `ukazatel` , na `nova_velikost` bajtů.
 - Může blok rozšířit na stávajícím místě, nebo jej přesunout jinam (v tom případě původní paměť uvolní a data zkopíruje do nového bloku).
 - V případě selhání vrátí `NULL` a původní blok zůstane nezměněn.
 - Pokud je `ukazatel` `NULL` , chová se jako `malloc` . Pokud je `nova_velikost` `0` , chová se jako `free` . `c int *rozsirene_pole = (int*) realloc(dynamicke_pole, 20 * sizeof(int)); if (rozsirene_pole == NULL) { /* chyba */ } else { dynamicke_pole = rozsirene_pole; }`
- `void free(void* ukazatel) :`
 - Uvolní dynamicky alokovanou paměť, na kterou ukazuje `ukazatel` .
 - Po uvolnění je paměť k dispozici operačnímu systému nebo pro další alokace.
 - Je kriticky důležité uvolnit veškerou alokovanou paměť, aby se předešlo únikům paměti. `c free(dynamicke_pole);`

Správa paměti a rizika

Na rozdíl od moderních jazyků (Java, C#, Python) nemá jazyk C automatický Garbage Collector. Vývojář je plně zodpovědný za životní cyklus alokované paměti.

- **Memory Leak (Únik paměti):**
 - Nastává, když dynamicky alokovaná paměť není uvolněna pomocí `free()`, a program ztratí odkaz na tuto paměť (např. ukazatel je přepsán nebo funkce skončí).
 - Program postupně spotřebovává víc a víc RAM, dokud ho operační systém neukončí nebo se systém nezpomalí.
- **Dangling Pointer (Zbloudilý ukazatel):**
 - Situace, kdy ukazatel stále míří na adresu paměti, která už ale byla uvolněna pomocí `free()`.
 - Přístup přes takový ukazatel vede k nedefinovanému chování (Undefined Behavior), pádu aplikace (Segmentation fault) nebo narušení integrity dat.
 - **Dobrá praxe:** Po `free(ptr)`; vždy nastavte `ptr = NULL`;
- **Double Free (Dvojitě uvolnění):**
 - Pokus o uvolnění paměti, která již byla uvolněna.
 - Vede k nedefinovanému chování, často k pádu programu nebo poškození haldy.
- **Buffer Overflow (Přetečení bufferu):**
 - Zápis dat za hranice alokovaného paměťového bloku (pole).
 - Může přepsat sousední data, způsobit pád programu nebo být zneužit k bezpečnostním útokům.
- **Kontrola návratové hodnoty:** Vždy je nutné kontrolovat, zda funkce pro alokaci paměti (`malloc`, `calloc`, `realloc`) vrátila `NULL`, což signalizuje selhání alokace.

Příklad dynamické alokace:

```
#include <stdio.h>
#include <stdlib.h> // Pro malloc a free
#include <string.h> // Pro strcpy
```

```

int main() {
    int n;
    printf("Zadej velikost pole: ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        fprintf(stderr, "Neplatná velikost pole.\n");
        return 1;
    }

    // Dynamická alokace pole o velikosti n prvků typu
    int
    int *dynamicke_pole = (int*) malloc(n * sizeof(int));

    if (dynamicke_pole == NULL) {
        // Kontrola, zda operační systém paměť úspěšně
        přidělil
        fprintf(stderr, "Chyba: Nepodařilo se alokovat
        paměť.\n");
        return 1;
    }

    // Práce s polem (úplně stejně jako s běžným polem)
    for (int i = 0; i < n; ++i) {
        dynamicke_pole[i] = i * 10;
        printf("dynamicke_pole[%d] = %d\n", i,
        dynamicke_pole[i]);
    }

    // Korektní uvolnění paměti na konci
    free(dynamicke_pole);
    dynamicke_pole = NULL; // Dobrá praxe proti
    zbloudilým ukazatelům

    // Pokus o přístup k uvolněné paměti (dangling
    pointer) by byl chyba!
    // printf("%d\n", dynamicke_pole[0]); // NEBEZPEČNÉ!

```

```
    return 0;  
}
```

Praktické použití

- **Nízkoúrovňové programování:** Vývoj ovladačů zařízení, operačních systémů, embedded systémů, kde je přímá kontrola nad pamětí nezbytná.
- **Implementace datových struktur:** Základ pro spojové seznamy, stromy, grafy, hashovací tabulky a další dynamické struktury.
- **Optimalizace výkonu:** Přímý přístup k paměti a manuální správa umožňuje jemnou optimalizaci pro kritické aplikace.
- **Práce s velkými datovými sadami:** Alokace paměti pro data, jejichž velikost není známa předem nebo přesahuje kapacitu zásobníku.
- **Vytváření komplexních datových modelů:** Struktury umožňují logické seskupení souvisejících dat do jednoho celku.
- **Komunikace s hardwarem:** Ukazatele se používají pro přímý přístup k paměťově mapovaným registrům zařízení.

Nejčastější chyby u zkoušky

- **Dereference neinicializovaného ukazatele:** Pokus o přístup k paměti, na kterou ukazatel neukazuje na platnou adresu.
- **Memory leak:** Zapomenutí uvolnit dynamicky alokovanou paměť pomocí `free()`.
- **Dangling pointer:** Použití ukazatele poté, co byla paměť, na kterou ukazoval, uvolněna.
- **Double free:** Pokus o uvolnění paměti, která již byla uvolněna.
- **Buffer overflow:** Zápis dat za hranice alokovaného pole nebo paměťového bloku.
- **Nekontrolování NULL návratové hodnoty:** Ignorování selhání alokace paměti z `malloc`, `calloc` nebo `realloc`.

- **Chybné použití operátorů `&` a `*`** : Záměna získání adresy a dereference.
- **Nepochopení ukazatelové aritmetiky**: Předpoklad, že `p + 1` vždy posune ukazatel o jeden bajt, místo o `sizeof(typ)` .
- **Záměna zásobníku a haldy**: Neznalost rozdílů v alokaci, životním cyklu a správě paměti.
- **Předpoklad pevných velikostí typů**: Tvrzení, že `int` má vždy 32 bitů nebo `char` vždy 8 bitů, místo pochopení, že jsou to minimální garantované velikosti, které se mohou lišit.

Paměťová pomůcka

"Ukazatel je adresa, pole je souvislá adresa. Stack je automatika, Heap je zodpovědnost."

Shrnutí kapitoly

Jazyk C poskytuje nízkoúrovňovou kontrolu nad pamětí pomocí základních a strukturovaných datových typů, polí a ukazatelů. Pole a ukazatele jsou úzce propojeny a umožňují efektivní manipulaci s daty. Dynamická alokace na haldě (`malloc` , `calloc` , `realloc` , `free`) umožňuje flexibilní správu paměti za běhu programu, ale vyžaduje pečlivou manuální správu, aby se předešlo chybám jako memory leaks, dangling pointers a double frees. Správné pochopení těchto konceptů je klíčové pro psaní robustního a bezpečného C kódu.

Typické zkuškové otázky

1. **Co je ukazatel a jak souvisí s adresou? Vysvětlete použití operátorů `&` a `*` .**

- **Odpověď**: Ukazatel je proměnná, jejíž hodnotou je paměťová adresa jiné proměnné. Jinými slovy, ukazatel "ukazuje" na místo v paměti, kde je uložena hodnota.

- **Operátor & (referenční operátor / "adresa z"):** Používá se k získání paměťové adresy proměnné. Například, `&x` vrátí adresu, kde je proměnná `x` uložena.
- **Operátor * (dereferenční operátor / "hodnota na adrese"):** Používá se k přístupu k hodnotě uložené na adrese, na kterou ukazatel ukazuje. Například, pokud `p` je ukazatel, `*p` vrátí hodnotu uloženou na adrese, kterou `p` obsahuje.

2. Jaký je rozdíl mezi zásobníkem (Stack) a haldou (Heap) z hlediska alokace paměti v C? Vysvětlete použití `malloc` a `free`.

○ Odpověď:

- **Zásobník (Stack):** Paměť alokovaná na zásobníku je spravována automaticky. Používá se pro lokální proměnné a parametry funkcí. Alokace a dealokace je velmi rychlá, probíhá při vstupu a výstupu z funkce/bloku. Velikost paměti musí být známá v době kompilace. Paměť je uvolněna automaticky po opuštění bloku, ve kterém byla alokována.
- **Halda (Heap):** Paměť alokovaná na haldě je spravována dynamicky programátorem. Používá se pro data, jejichž velikost není známa předem nebo která musí přežít životnost funkce. Alokace je pomalejší než na zásobníku. Programátor je zodpovědný za explicitní uvolnění paměti pomocí `free()`.
- **`malloc(size_t size)`:** Funkce pro dynamickou alokaci paměti na haldě. Alokuje blok paměti o velikosti `size` bajtů a vrací `void*` ukazatel na začátek alokovaného bloku. Alokovaná paměť není inicializována. V případě selhání vrací `NULL`.
- **`free(void* ptr)`:** Funkce pro uvolnění dříve dynamicky alokované paměti, na kterou ukazuje `ptr`. Po volání `free` je paměť k dispozici pro další použití, a ukazatel `ptr` se stává zbloudilým (dangling pointer), dokud není nastaven na `NULL`.

3. **Vysvětlete koncept "array-pointer duality" v jazyce C a uveďte, jak funguje ukazatelová aritmetika s poli.**

- **Odpověď:** "Array-pointer duality" (dualita pole-ukazatel) znamená, že v mnoha kontextech (zejména ve výrazech a při předávání funkcím) se jméno pole automaticky "rozpadne" (decays) na ukazatel na svůj první prvek. To znamená, že pole se chová jako `&pole[0]`.
- **Ukazatelová aritmetika s poli:** Když provádíme aritmetické operace s ukazatelem, který ukazuje na prvek pole, operace se provádí s ohledem na velikost datového typu, na který ukazatel ukazuje.
 - Například, pokud `p` je ukazatel typu `int*` a ukazuje na `pole[i]`, pak `p + 1` neposune ukazatel o jeden bajt, ale o `sizeof(int)` bajtů, takže bude ukazovat na `pole[i+1]`.
 - Podobně, `p - 1` posune ukazatel na `pole[i-1]`.
 - Zápis `pole[i]` je syntaktický cukr pro `*(pole + i)`. To ukazuje, že indexování pole je ve skutečnosti realizováno pomocí ukazatelové aritmetiky a dereference.

4. **Popište, co jsou to strukturované datové typy (struct , union) a jaký je mezi nimi hlavní rozdíl.**

- **Odpověď:** Strukturované datové typy umožňují seskupit více proměnných, potenciálně různých typů, pod jedno společné jméno.
 - **struct (struktura):** Seskupuje několik členů, které jsou uloženy v paměti sekvenčně, jeden za druhým. Každý člen má své vlastní paměťové místo. Celková velikost struktury je součet velikostí jejích členů (plus případný padding pro zarovnání). Struktura se používá, když chceme uchovávat sadu souvisejících, ale nezávislých datových položek.
 - **union (unie):** Seskupuje několik členů, ale všechny členy sdílejí stejné paměťové místo. Velikost unie je rovna velikosti největšího člena. V daný okamžik může unie uchovávat hodnotu pouze jednoho ze svých členů. Unie se

používá, když chceme efektivně využít paměť pro data, která se vzájemně vylučují, nebo když chceme interpretovat stejná binární data různými způsoby.

- **Hlavní rozdíl:** V `struct` mají všechny členy svá vlastní paměťová místa a jsou přístupné současně. V `union` sdílejí všechny členy stejné paměťové místo, a v daný okamžik je platná pouze hodnota jednoho člena.

5. Vysvětlete pojmy "Memory Leak" a "Dangling Pointer" a jak se jim v C kódu bránit.

○ Odpověď:

- **Memory Leak (Únik paměti):** Nastává, když program dynamicky alokuje paměť na haldě (např. pomocí `malloc`), ale tuto paměť nikdy neuvolní pomocí `free()`, a zároveň ztratí odkaz na ni (např. přepsáním ukazatele nebo ukončením funkce, ve které byl ukazatel lokální). Tato paměť zůstává obsazená a nedostupná pro další použití, což vede k postupnému vyčerpávání systémových zdrojů.
- **Dangling Pointer (Zbloudilý ukazatel):** Je to ukazatel, který stále obsahuje adresu paměti, která však již byla uvolněna (např. voláním `free()`). Pokus o dereferenci nebo použití takového ukazatele vede k nedefinovanému chování, které může zahrnovat pád programu, poškození dat nebo bezpečnostní zranitelnosti.
- **Jak se bránit:**
 - **Proti Memory Leaks:** Vždy uvolněte dynamicky alokovanou paměť pomocí `free()`, jakmile ji již nepotřebujete. Zajistěte, aby každé `malloc` (nebo `calloc`, `realloc`) mělo odpovídající `free`.
 - **Proti Dangling Pointers:** Po uvolnění paměti pomocí `free(ptr)`; okamžitě nastavte ukazatel na `NULL` (tj. `ptr = NULL`). Tím se zabrání neúmyslnému použití zbloudilého ukazatele, protože dereference `NULL` ukazatele obvykle vede k okamžitému pádu

programu (segmentation fault), což je snazší
detekovat a opravit než nedefinované chování.

Níže je připravený studijní materiál k okruhu "12. Algoritmy pro vyhledávání a řazení, složitost algoritmů" doplněný a naformátovaný dle vašich přísných pravidel.

Kapitola 12: Algoritmy pro vyhledávání a řazení, složitost algoritmů

Klíčové pojmy

- algoritmus
- časová složitost
- paměťová složitost
- Big-O notace
- lineární vyhledávání
- binární vyhledávání
- quicksort
- mergesort
- heapsort
- stabilita řazení
- in-place algoritmus
- pivot
- halda

A. Složitost algoritmů (Asymptotická složitost)

Složitost algoritmů nám umožňuje porovnávat efektivitu různých algoritmů nezávisle na konkrétním hardwaru, operačním systému nebo použitém

programovacím jazyce. Sleduje se, jak roste spotřeba zdrojů (čas nebo paměť) v závislosti na růstu velikosti vstupních dat n .

Složitost dělíme na dvě složky:

- **Časová složitost:** Doba běhu algoritmu, vyjádřená počtem provedených elementárních operací (např. porovnání, přiřazení, aritmetické operace).
- **Prostorová (paměťová) složitost:** Množství dodatečné paměti RAM, kterou algoritmus ke svému běhu potřebuje, kromě paměti pro vstupní data.

Velká O notace (Asymptotická horní mez)

Značí se jako $O(f(n))$ a definuje asymptotickou horní mez pro růst funkce, tedy nejhorší možný případ (Worst-case), jaký může nastat. Formálně, funkce $g(n)$ je $O(f(n))$, pokud existují kladné konstanty c a n_0 takové, že pro všechna $n \geq n_0$ platí $0 \leq g(n) \leq c \cdot f(n)$. V Big-O notaci se zanedbávají aditivní konstanty a méně významné členy (např. polynom $3n^2 + 5n + 10$ má složitost $O(n^2)$), protože se zaměřujeme na růstovou rychlost pro velké n .

Přehled základních tříd složitosti (od nejrychlejších po nejpomalejší)

- $O(1)$ – **Konstantní:** Doba běhu nebo spotřeba paměti nezávisí na velikosti dat n .
 - **Příklad:** Přístup k prvku pole podle indexu, vložení prvku na začátek spojového seznamu.
- $O(\log n)$ – **Logaritmická:** Doba běhu nebo spotřeba paměti roste logaritmicky s velikostí dat. Velmi efektivní, při každém kroku se velikost dat zmenší o konstantní faktor (např. o polovinu).
 - **Příklad:** Binární vyhledávání v setříděném poli.
- $O(n)$ – **Lineární:** Doba běhu nebo spotřeba paměti roste přímo úměrně s velikostí dat n .

- **Příklad:** Lineární vyhledávání v neseřazeném poli, průchod celým spojovým seznamem.
- $O(n \log n)$ – **Lineárnitmetická:** Doba běhu nebo spotřeba paměti roste n krát logaritmus n . Typická pro nejlepší obecné řadící algoritmy.
 - **Příklad:** Merge Sort, Quick Sort (průměrný případ), Heap Sort.
- $O(n^2)$ – **Kvadratická:** Doba běhu nebo spotřeba paměti roste s druhou mocninou velikosti dat n . Typická pro jednoduché, neefektivní řadící algoritmy se dvěma vnořenými cykly.
 - **Příklad:** Bubble Sort, Insertion Sort, Selection Sort.
- $O(2^n)$ – **Exponenciální:** Doba běhu nebo spotřeba paměti roste exponenciálně s velikostí dat n . Výpočetně extrémně náročné, prakticky použitelné jen pro velmi malá n .
 - **Příklad:** Naivní rekurzivní výpočet Fibonacciho čísel, problém obchodního cestujícího (brute-force).

B. Algoritmy pro vyhledávání

Úkolem je najít pozici hledaného prvku (klíče) v datové struktuře.

1. Sekvenční (lineární) vyhledávání

- **Princip:** Prochází pole prvek po prvku od začátku do konce, dokud hledanou hodnotu nenajde nebo nedosáhne konce pole.
- **Podmínka:** Data mohou být v libovolném (i chaotickém) pořadí.
- **Časová složitost:**
 - Nejhorší případ: $O(n)$ (prvek není nalezen nebo je na konci pole).
 - Průměrný případ: $O(n)$.
 - Nejlepší případ: $O(1)$ (prvek je na začátku pole).

2. Binární vyhledávání (Metoda půlení intervalu)

- **Princip:** Podívá se na prvek přesně uprostřed setříděného pole. Pokud je hledaný prvek menší než prostřední, zahodí pravou polovinu pole a pokračuje hledáním v levé polovině. Pokud je větší, zahodí levou polovinu. Tento postup opakuje, dokud prvek nenajde nebo se interval hledání nestane prázdným.
- **Podmínka:** Data musí být striktně setříděná.
- **Časová složitost:**
 - Nejhorší případ: $O(\log n)$.
 - Průměrný případ: $O(\log n)$.
 - Nejlepší případ: $O(1)$ (prvek je uprostřed pole).
- **Příklad (pseudokód):**

```
c int binsearch (int a[], int n, int x) {
    int l = 0, r = n - 1;
    while (l <= r) {
        int m = l + (r - l) / 2;
        // Zamezí přetečení oproti (l+r)/2
        if (a[m] == x) return m;
        if (a[m] < x) l = m + 1;
        else r = m - 1;
    }
    return -1; // Prvek nebyl nalezen
}
```

 - `l` (left) a `r` (right) definují aktuální interval hledání.
 - `m` (middle) je index prostředního prvku.
 - Cyklus pokračuje, dokud se `l` nepřekříží s `r`.

C. Algoritmy pro řazení (Třídění)

Cílem je přeskládat prvky v poli podle definovaného klíče (např. vzestupně).

U algoritmů rozlišujeme dvě důležité vlastnosti:

- **Stabilita:** Stabilní algoritmus zachová původní vzájemné pořadí prvků se stejnou hodnotou klíče. To je důležité, pokud mají prvky kromě klíče i další data a původní pořadí těchto "stejných" prvků je relevantní.
- **In-place (Na místě):** Algoritmus nepotřebuje k seřazení dodatečné pole nebo potřebuje jen konstantní množství dodatečné paměti $O(1)$ (kromě rekursivního zásobníku).

1. Jednoduché algoritmy (Kvadratické, $O(n^2)$)

Tyto algoritmy jsou vhodné pouze pro malé množství dat, protože jejich časová náročnost roste kvadraticky.

- **Select Sort (Řazení výběrem):**

- **Princip:** Najde nejmenší (nebo největší) prvek v neseříděné části pole a zamění ho s prvkem na aktuální pozici seříděné části. Tento postup opakuje, dokud není celé pole seříděno.
- **Vlastnosti:** Nestabilní, In-place.
- **Složitost:** Vždy $O(n^2)$ (jak nejhorší, tak průměrný případ).

- **Insert Sort (Řazení vkládáním):**

- **Princip:** Rozdělí pole na seříděnou a neseříděnou část. Bere postupně prvky z neseříděné části a "vkládá" je na správné místo v seříděné části posouváním větších prvků doprava.
- **Vlastnosti:** Stabilní, In-place.
- **Složitost:**
 - Nejhorší případ: $O(n^2)$ (pole je seřazeno v opačném pořadí).
 - Průměrný případ: $O(n^2)$.
 - Nejlepší případ: $O(n)$ (pole je již seříděné) – velmi efektivní pro doplňování prvků do již téměř seříděných dat.

- **Bubble Sort (Bublínkové řazení):**

- **Princip:** Opakovaně prochází pole a porovnává dva sousední prvky. Pokud jsou v nesprávném pořadí, prohodí je. Větší prvky tak postupně "probublávají" na konec pole.
- **Vlastnosti:** Stabilní, In-place.
- **Složitost:**
 - Nejhorší případ: $O(n^2)$.
 - Průměrný případ: $O(n^2)$.
 - Nejlepší případ: $O(n)$ (pole je již seříděné, pokud je implementována optimalizace pro detekci průchodu bez výměn).

- **Poznámka:** Výpočetně nejméně efektivní z jednoduchých algoritmů pro obecný případ.

2. Pokročilé algoritmy ($O(n \log n)$)

Využívají strategii Rozděl a panuj (Divide and Conquer) a jsou standardem v praxi pro větší objemy dat.

- **Quick Sort (Rychlé řazení):**

- **Princip:** Zvolí se jeden prvek jako tzv. **pivot**. Pole se přeuspořádá tak, že všechny prvky menší než pivot se přesunou vlevo od něj a všechny větší vpravo. Pivot je tak na své konečné pozici. Poté se rekurzivně stejný postup aplikuje na levou a pravou část pole.
- **Vlastnosti:** Nestabilní, In-place (s drobnou rekurzivní režii na zásobníku, která je $O(\log n)$ v průměrném případě, ale $O(n)$ v nejhorším).
- **Složitost:**
 - Průměrný případ: $O(n \log n)$.
 - Nejhorší případ: $O(n^2)$ (pokud je zvolen špatný pivot, např. vždy nejmenší/největší prvek u již setříděného pole).
- **Poznámka:** V praxi je často nejrychlejší díky nízké konstantě a dobrému využití cache.

- **Merge Sort (Řazení slučováním):**

- **Princip:** Pole se rekurzivně dělí na poloviny, dokud nezbudou jednotlivé jednoprvkové úseky. Tyto úseky jsou již setříděné. Poté se tyto setříděné úseky slévají (slučují) zpět dohromady tak, že se při slévání rovnou správně zařazují.
- **Vlastnosti:** Stabilní, NENÍ In-place (vyžaduje pomocné pole o velikosti $O(n)$ pro slévání).
- **Složitost:** Garantovaná $O(n \log n)$ za všech okolností (nejhorší, průměrný i nejlepší případ).
- **Poznámka:** Vhodný pro externí řazení (data se nevejdou do paměti) a paralelní zpracování.

- **Heap Sort (Řazení haldou):**

- **Princip:** Prvky pole se nejprve uspořádají do datové struktury zvané **binární halda** (konkrétně max-halda, kde rodič je vždy větší nebo roven svým potomkům). Největší prvek z vrcholu haldy (kořen) se opakovaně odebírá (zamění se s posledním prvkem haldy) a ukládá na konec pole. Halda se poté znovu uspořádá.
- **Vlastnosti:** Nestabilní, In-place.
- **Složitost:** Garantovaná $O(n \log n)$ za všech okolností.
- **Poznámka:** Dobrá volba, pokud je vyžadována garantovaná $O(n \log n)$ složitost a in-place řazení.

Nejčastější chyby u zkoušky

- Hodnocení algoritmu jen podle jednoho malého vstupu.
- Použití binárního vyhledávání na neseřazená data.
- Záměna stability a rychlosti řazení.
- Neznalost rozdílu mezi časovou a paměťovou složitostí.
- Neznalost dopadu volby pivota u Quick Sortu.

Typické zkouškové otázky

1. Vysvětlete notaci Big-O.

- **Odpověď:** Notace Big-O (Velké O) je matematický zápis, který popisuje asymptotickou horní mez růstu funkce, typicky používaný pro vyjádření časové nebo paměťové složitosti algoritmů. Udává, jak se doba běhu nebo spotřeba paměti algoritmu chová pro dostatečně velké vstupní datové sady n . Zanedbává konstanty a méně významné členy, zaměřuje se na dominantní člen, který určuje rychlost růstu. Například $O(n^2)$ znamená, že složitost roste kvadraticky s velikostí vstupu.

2. Porovnejte quicksort a mergesort.

- **Odpověď:**

- **Složitost:** Oba mají průměrnou časovou složitost $O(n \log n)$. Quick Sort má v nejhorším případě $O(n^2)$, zatímco Merge Sort má garantovanou $O(n \log n)$ ve všech případech.
- **Paměťová složitost:** Quick Sort je obvykle in-place (vyžaduje $O(\log n)$ paměti pro rekursivní zásobník v průměrném případě, $O(n)$ v nejhorším). Merge Sort není in-place a vyžaduje $O(n)$ dodatečné paměti pro pomocné pole při slučování.
- **Stabilita:** Quick Sort je nestabilní. Merge Sort je stabilní.
- **Praktické použití:** Quick Sort je v praxi často rychlejší díky menší konstantě a lepšímu využití cache, ale závisí na dobré volbě pivotu. Merge Sort je vhodný pro externí řazení a paralelní zpracování, kde je stabilita a garantovaná složitost důležitá.

3. Kdy lze použít binární vyhledávání?

- **Odpověď:** Binární vyhledávání lze použít pouze v případě, že jsou data, ve kterých se vyhledává, **striktně setříděná** (vzestupně nebo sestupně). Pokud data nejsou setříděná, je nutné je nejprve setřídít, nebo použít lineární vyhledávání.

4. Vysvětlete pojmy "stabilita" a "in-place" u řadicích algoritmů.

- **Odpověď:**
 - **Stabilita:** Řadicí algoritmus je stabilní, pokud zachovává původní relativní pořadí prvků, které mají stejnou hodnotu klíče. Pokud například máme seznam studentů se stejným příjmením, stabilní algoritmus zachová jejich původní pořadí (např. podle data narození), i když je řadíme podle příjmení.
 - **In-place:** Řadicí algoritmus je in-place, pokud k provedení řazení nepotřebuje významné množství dodatečné paměti, tj. jeho prostorová složitost je $O(1)$ (nebo $O(\log n)$ pro rekursivní algoritmy, které používají zásobník). Data jsou řazena přímo v původním paměťovém prostoru.

5. Uveďte příklad algoritmu s $O(n^2)$ a $O(n \log n)$ časovou složitostí a vysvětlete, proč se liší.

○ **Odpověď:**

- $O(n^2)$: Příkladem je **Bubble Sort**. Jeho složitost je $O(n^2)$, protože pro každou z n iterací (průchodů polem) musí v nejhorším případě provést až $n - 1$ porovnání a případných výměn. To vede k dvojici vnořených cyklů, kde každý cyklus prochází téměř celé pole.
- $O(n \log n)$: Příkladem je **Merge Sort**. Jeho složitost je $O(n \log n)$, protože používá strategii "rozděl a panuj". Pole je rekurzivně děleno na poloviny, což vytváří $\log n$ úrovní dělení. Na každé úrovni se provádí operace slučování (merge), která vyžaduje lineární čas $O(n)$ pro sloučení všech prvků. Kombinace $\log n$ úrovní a $O(n)$ práce na každé úrovni dává $O(n \log n)$. Rozdíl spočívá v efektivnějším přístupu k řešení problému pro velké n , kdy se problém rozkládá na menší, snadněji řešitelné podproblémy.

Shrnutí kapitoly

Znalost složitosti pomáhá odhadnout, zda algoritmus obstojí pro velká data. Volba algoritmu závisí na vlastnostech vstupu a požadavcích na paměť a stabilitu. Vyhledávací algoritmy se liší především požadavkem na setříděnost dat a z toho plynoucí složitostí. Řadicí algoritmy se dělí na jednoduché (kvadratické) a pokročilé (lineární), přičemž ty pokročilé jsou pro větší datové sady výrazně efektivnější. Důležité je také zvážit stabilitu a paměťové nároky algoritmu.

Zde je přepracovaný a doplněný studijní materiál k okruhu "13. Rekurse a její použití. Rekurzivní a nerekurzivní realizace vybraných algoritmů. Využití zásobníku programu."

13. Rekurse a zásobník programu

Klíčové pojmy

- rekurse
- rekurzivní volání
- báze rekurse
- rekurzivní krok
- zásobník programu (call stack)
- aktivační záznam (stack frame)
- návratová adresa
- stack overflow
- iterace
- koncová rekurse (tail recursion)
- optimalizace koncové rekurse (tail call optimization)

A. Pojem rekurse a její použití

Rekurse je programovací technika, při které se funkce volá sama. Může jít o **přímou rekurzi**, kdy funkce volá přímo sebe, nebo o **nepřímou (vzájemnou) rekurzi**, kdy funkce A volá funkci B, která následně volá funkci A (nebo jinou funkci, která vede zpět k A).

Aby rekurzivní algoritmus fungoval správně a neskončil nekonečným cyklem, musí splňovat dvě základní podmínky:

- **Bázový (kotevní) případ:** Podmínka, při které je výsledek znám přímo a funkce se již rekurzivně nevolá. Toto je ukončovací podmínka

rekurze.

- **Rekurzivní krok:** Krok, v němž funkce volá samu sebe, ale s modifikovaným (obvykle menším nebo jednodušším) vstupem, který se prokazatelně přibližuje k báзовému případu. Bez tohoto kroku by se rekurze nikdy neukončila.

Typické použití rekurze:

- **Hierarchické a stromové struktury:**
 - Procházení adresářové struktury na disku.
 - Práce s binárními vyhledávacími stromy (např. metody In-order, Pre-order, Post-order traversal).
 - Parsování složitých datových struktur jako XML/JSON.
 - Výpočet hloubky stromu.
- **Algoritmy typu Rozděl a panuj (Divide and Conquer):**
 - Třídící algoritmy jako Quick Sort, Merge Sort.
 - Binární vyhledávání.
 - Výpočet mocniny x^n efektivně.
- **Kombinatorické úlohy a prohledávání stavového prostoru:**
 - Generování permutací, kombinací.
 - Problém Hanojských věží.
 - Algoritmy Backtracking (např. řešení bludiště, problém osmi dam na šachovnici, řešení Sudoku).
 - Výpočet Fibonacciho posloupnosti.

Výhody rekurze:

- **Čitelnost a elegance:** Pro některé problémy (zejména ty s rekurzivní definicí) je rekurzivní řešení přirozenější a snáze pochopitelné.
- **Kratší kód:** Často vede ke kratšímu a kompaktnějšímu kódu.

Nevýhody rekurze:

- **Výkon:** Často je pomalejší než iterativní řešení kvůli režii volání funkcí a manipulaci se zásobníkem.
- **Paměťová náročnost:** Může spotřebovat hodně paměti na zásobníku, což vede k riziku přetečení zásobníku.
- **Ladění:** Může být obtížnější ladit kvůli hlubokému zanoření volání.

B. Využití zásobníku programu (Call Stack)

Rekurze je plně závislá na programovém zásobníku (Call Stack). Zásobník je datová struktura typu LIFO (Last-In, First-Out), která spravuje volání funkcí.

Mechanismus rekurzivního zanořování:

1. **Volání funkce:** Kdykoliv je zavolána jakákoliv funkce (včetně té rekurzivní), aktuální stav vykonávání se pozastaví.
2. **Vytvoření aktivačního záznamu (Stack Frame):** Na vrcholu zásobníku se vytvoří nový aktivační záznam (neboli zásobníkový rámeček). Tento záznam obsahuje klíčové informace pro správné fungování funkce a návrat z ní:
 - **Lokální proměnné:** Hodnoty všech lokálních proměnných deklarovaných v dané instanci funkce.
 - **Parametry funkce:** Hodnoty argumentů, které byly funkci předány.
 - **Návratová adresa:** Adresa v kódu, kam se má procesor vrátit po dokončení aktuální funkce.
 - **Uložené registry:** Hodnoty některých registrů procesoru, které je potřeba po návratu obnovit, aby se předchozí funkce mohla správně obnovit.
3. **Rekurzivní volání:** Pokud funkce v rámci svého těla provede další rekurzivní volání, celý proces se opakuje: aktuální funkce se pozastaví, na zásobník se přidá nový aktivační záznam pro nové volání atd. Paměť na zásobníku tak lineárně roste s hloubkou rekurze.
4. **Návrat z funkce:** Jakmile algoritmus konečně narazí na bázevý případ a funkce spočítá svůj výsledek, její aktivační záznam se ze zásobníku odstraní (tzv. "pop" operace). Řízení se vrátí na návratovou adresu uloženou v předchozím aktivačním záznamu, a předchozí funkce může pokračovat ve svém vykonávání. Záznamy se postupně odstraňují, dokud se nedopočítá původní volání.

Riziko: Stack Overflow (Přetečení zásobníku)

Zásobník programu má pevně vymezenou velikost, která je obvykle řádově v jednotkách megabajtů (např. 1 MB, 8 MB). Pokud je rekurze příliš hluboká

(např. chybí bazový případ, nebo zpracováváte velmi velkou datovou strukturu pomocí naivní rekurze), zásobník spotřebuje veškerou přidělenou paměť. Dojde k havárii programu na chybu **Stack Overflow**. V jazyce C se to často projevuje jako `Segmentation fault` nebo podobná chyba přístupu k paměti.

C. Rekurzivní vs. nerekurzivní (iterativní) realizace algoritmů

Každý rekurzivní algoritmus lze teoreticky přepsat do nerekurzivní (iterativní) podoby pomocí cyklů (`while`, `for`). Volba mezi rekurzivním a iterativním přístupem závisí na konkrétním problému, požadavcích na výkon, paměť a čitelnost kódu.

Převod rekurze na iteraci:

Pokud je k vyřešení úlohy potřeba ukládat historii stavů nebo simulovat volání funkcí (např. pro algoritmy prohledávání do hloubky), iterativní verze si musí interně vytvořit a spravovat vlastní datovou strukturu, která funguje jako zásobník (např. pole nebo spojový seznam alokovaný na haldě). Tím se sice vyhneme přetečení *programového* zásobníku, ale paměťová náročnost se přesune na *haldu*.

Srovnání přístupů:

Vlastnost	Rekurzivní realizace	Iterativní realizace
Čitelnost	Často přirozenější a elegantnější pro rekurzivní problémy	Může být složitější pro rekurzivně definované problémy
Výkon	Vyšší režie kvůli volání funkcí a správě zásobníku	Obvykle rychlejší, nižší režie volání funkcí
Paměť	Využívá programový zásobník (stack), riziko stack overflow	Využívá haldu (heap) pro explicitní zásobník, menší riziko overflow

Vlastnost	Rekurzivní realizace	Iterativní realizace
Složitost kódu	Často kratší	Může být delší a vyžadovat explicitní správu stavu
Ladění	Může být obtížnější kvůli hlubokému zanoření	Obvykle snazší, přímější tok řízení

Příklad: Výpočet faktoriálu $n!$ v jazyce C

Faktoriál $n!$ je definován jako $n \times (n - 1) \times \dots \times 1$, přičemž $0! = 1$.

1. Rekurzivní realizace:

```
c long faktorial_rekurzivni(int n) { // Bázový případ:
    Pokud n je 0 nebo 1, faktoriál je 1.
    if (n <= 1) {
        return 1; } // Rekurzivní krok: n * faktoriál(n-1)
    return n * faktorial_rekurzivni(n - 1); }
```

- **Vysvětlení:** Funkce se volá sama se zmenšeným argumentem $n-1$, dokud n nedosáhne 1. Poté se výsledky postupně násobí zpět nahoru.
- **Riziko:** Při volání pro velké n (např. $n=10000$) hrozí pád na přetečení zásobníku, protože se vytvoří příliš mnoho aktivačních záznamů.

2. Iterativní (nerekurzivní) realizace:

```
c long faktorial_iterativni(int n) { long vysledek = 1;
    // Pro záporná čísla faktoriál není definován, pro 0! a
    1! je 1.
    if (n < 0) { // Zde by se měla řešit chyba,
        např. vrátit -1 nebo vyhodit výjimku
        return -1; } //
    Cyklus od 2 do n, násobí výsledek postupně
    for (int i = 2; i <= n; i++) { vysledek *= i; }
    return vysledek; }
```

- **Vysvětlení:** Funkce používá jednoduchý cyklus `for` k postupnému násobení čísel.

- **Paměťová složitost:** Je $O(1)$, protože alokuje pouze konstantní počet proměnných bez ohledu na n . Nehrozí přetečení zásobníku.
- **Výhoda:** Je robustnější pro velké vstupy z hlediska paměti.

Pokročilý koncept: Koncová rekurze (Tail Recursion)

Koncová rekurze je speciální typ rekurze, kde je rekurzivní volání **úplně poslední operací**, kterou funkce provádí. To znamená, že výsledek rekurzivního volání se už nijak dál neupravuje (např. se k němu nic nepřičítá, nenásobí se jím apod.).

Příklad koncové rekurze (faktoriál s akumulátorem):

```
long faktorial_koncova_rekurze_pomocna(int n, long
akumulator) {
    if (n <= 1) {
        return akumulator; // Bázový případ, vrací
akumulovanou hodnotu
    }
    // Rekurzivní volání je poslední operací, výsledek se
přímo vrací
    return faktorial_koncova_rekurze_pomocna(n - 1,
akumulator * n);
}

long faktorial_koncova_rekurze(int n) {
    if (n < 0) {
        return -1; // Ošetření záporných čísel
    }
    return faktorial_koncova_rekurze_pomocna(n, 1); //
Počáteční volání s akumulátorem 1
}
```

Optimalizace koncové rekurze (Tail Call Optimization - TCO):

Moderní kompilátory (např. GCC, Clang s optimalizačními flagy) dokážou koncovou rekurzi optimalizovat. Místo vytváření nového aktivačního záznamu pro každé rekurzivní volání, kompilátor přepíše hodnoty parametrů v aktuálním záznamu a skočí na začátek funkce. Fakticky ji tedy převede na běžný cyklus. Tím se eliminuje režie volání funkcí a riziko přetečení zásobníku, což kombinuje eleganci rekurze s efektivitou iterace. TCO však není garantováno ve všech jazycích a kompilátorech (např. C++ standard TCO nevyžaduje, ale mnoho kompilátorů ji implementuje; Java ji nemá).

Shrnutí kapitoly

Rekurze je mocná programovací technika, která umožňuje elegantně řešit problémy s rekurzivní strukturou. Její fungování je úzce spjato se zásobníkem programu, který ukládá aktivační záznamy pro každé volání funkce. Hluboká rekurze může vést k přetečení zásobníku. Každý rekurzivní algoritmus lze převést na iterativní, což často zlepšuje výkon a paměťovou efektivitu, zejména pro velké vstupy. Koncová rekurze představuje speciální případ, který může být optimalizován kompilátorem na iterativní kód, čímž se minimalizují nevýhody rekurze.

Typické zkouškové otázky

1. **Definujte rekurzi a vysvětlete její základní principy. Uveďte příklad problému, který je přirozeně řešitelný rekurzí.**

Odpověď: Rekurse je programovací technika, při které funkce volá sama sebe (přímo nebo nepřímo). Její základní principy jsou:

- **Bázový (kotevní) případ:** Podmínka, při které se rekurze zastaví a vrátí přímý výsledek. Je to ukončovací podmínka.
- **Rekurzivní krok:** Funkce volá sebe sama s modifikovaným vstupem, který se postupně přibližuje k báзовému případu. Příkladem problému přirozeně řešitelného rekurzí je **výpočet faktoriálu** ($n! = n \times (n - 1)!$ s báзовým případem $0! = 1$) nebo **procházení stromové struktury** (např. binární vyhledávací strom).

2. **Popište, jak rekurze využívá zásobník programu (call stack). Co je aktivační záznam a jaké informace obsahuje? Jaké riziko je s tímto mechanismem spojeno?**

Odpověď: Při každém volání rekurzivní funkce se na vrchol zásobníku programu (call stack) přidá nový **aktivační záznam (stack frame)**.

Tento záznam obsahuje:

- Lokální proměnné aktuální instance funkce.
- Parametry předané funkci.
- Návratovou adresu (kam se má program vrátit po dokončení funkce).
- Uložené hodnoty registrů procesoru. S každým dalším rekurzivním voláním se na zásobník přidává další aktivační záznam, což způsobuje lineární růst paměti spotřebované zásobníkem. Hlavním rizikem je **Stack Overflow (přetečení zásobníku)**. Pokud je rekurze příliš hluboká (např. chybí bazový případ nebo je vstupní data příliš velká), zásobník vyčerpá přidělenou paměť a program havaruje.

3. **Vysvětlete rozdíl mezi rekurzivní a iterativní realizací algoritmu. Kdy je vhodnější použít jeden přístup před druhým?**

Odpověď:

- **Rekurzivní realizace** řeší problém voláním sebe sama, spoléhá na zásobník programu pro správu stavu a návratových adres. Kód je často elegantnější a přirozenější pro rekurzivně definované problémy.
- **Iterativní realizace** řeší problém pomocí cyklů (např. `for`, `while`) a explicitně spravuje svůj stav pomocí proměnných nebo vlastních datových struktur. Je obvykle paměťově efektivnější a rychlejší, protože se vyhýbá režii volání funkcí. **Kdy je co vhodnější:**
 - **Rekurze je vhodnější**, když je problém přirozeně rekurzivní (např. procházení stromů, algoritmy rozděl a panuj), což vede k čistšímu a čitelnějšímu kódu.
 - **Iterace je vhodnější**, když je hloubka rekurze potenciálně velmi velká (aby se předešlo Stack Overflow), nebo když je kritický výkon a paměťová efektivita. Také pro jednoduché lineární problémy (jako faktoriál) je iterace často jasnější a efektivnější.

4. **Co je koncová rekurze a jaká je její hlavní výhoda?**

Odpověď: Koncová rekurze (Tail Recursion) je speciální forma rekurze, kde je rekurzivní volání **poslední operací**, kterou funkce provádí. To znamená, že výsledek rekurzivního volání se už nijak dál nezpracovává (např. nepřičítá se k němu žádná hodnota). Její hlavní výhodou je, že moderní kompilátory mohou provést **optimalizaci koncové rekurze (Tail Call Optimization - TCO)**. Při TCO kompilátor nepřidává nový aktivační záznam na zásobník, ale místo toho přepíše parametry v aktuálním záznamu a provede skok na začátek funkce. Tím se rekurze efektivně převede na iterativní kód, což eliminuje režii volání funkcí a riziko přetečení zásobníku, aniž by se ztratila čitelnost rekurzivního zápisu.

14. Členění programu v jazyce vyšší úrovně. Metody, funkce, procedury, makra. Parametry metod, procedur a funkcí a způsoby jejich předávání. Globální a lokální proměnné.

Klíčové pojmy

- **Modularita:** Rozdělení programu na menší, samostatné a znovupoužitelné části.
- **Funkce:** Podprogram, který přijímá vstupy, provádí výpočet a vrací hodnotu.
- **Procedura:** Podprogram, který provádí posloupnost příkazů, ale nevrací explicitně hodnotu (často má vedlejší efekty).
- **Metoda:** Funkce přidružená k objektu nebo třídě v OOP.
- **Makro:** Textová substituce prováděná preprocesorem před kompilací.
- **Parametry:** Vstupy předávané podprogramům.
- **Předávání hodnotou (Call by Value):** Předání kopie hodnoty parametru.
- **Předávání odkazem (Call by Reference):** Předání adresy paměti, kde je parametr uložen.
- **Lokální proměnná:** Proměnná viditelná a existující pouze v rámci bloku, kde je definována.
- **Globální proměnná:** Proměnná viditelná a existující v celém programu nebo v rámci celého souboru.
- **Rozsah platnosti (Scope):** Oblast kódu, kde je proměnná přístupná.
- **Životní cyklus (Lifetime):** Doba, po kterou proměnná existuje v paměti.

- **Zásobník (Stack):** Paměťová oblast pro lokální proměnné a návratové adresy.
 - **Halda (Heap):** Paměťová oblast pro dynamicky alokované proměnné.
 - **Vedlejší efekt (Side Effect):** Změna stavu mimo rozsah funkce (např. změna globální proměnné, I/O operace).
-

A. Členění programu (Modularita)

Větší a složitější programy nelze efektivně psát jako jeden monolitický blok kódu. Místo toho se člení do menších, logicky ucelených a znovupoužitelných celků, což se nazývá **modularita**. Tento přístup přináší řadu výhod:

- **Zvýšená přehlednost a čitelnost:** Menší moduly se snáze chápou a udržují.
- **Usnadnění testování:** Jednotlivé moduly lze testovat nezávisle.
- **Zamezení duplicitě kódu (DRY princip):** Znovupoužitelné části kódu se píšou jen jednou.
- **Umožnění spolupráce:** Více vývojářů může pracovat na různých modulech současně.
- **Abstrakce a skrývání detailů:** Moduly mohou skrývat svou vnitřní implementaci a vystavovat pouze jasné rozhraní.
- **Snadnější údržba a modifikace:** Změny v jednom modulu mají menší dopad na zbytek programu.

Základní podprogramové jednotky jsou:

Funkce

- **Definice:** Nezávislý blok kódu, který je navržen k provedení specifického úkolu. Přijímá vstupy, nazývané **parametry**, provede výpočet a **vrací hodnotu** jako svůj výsledek.
- **Návratová hodnota:** Hodnota je vrácena pomocí klíčového slova `return`. Typ návratové hodnoty je definován v hlavičce funkce.
- **Typické použití:** Výpočty (např. matematické funkce), transformace dat. Ideální funkce by měla být "čistá" – pro stejné vstupy vždy vrátí

stejný výstup a nemít žádné vedlejší efekty.

- **Příklad (C):** `c int soucet(int a, int b) { return a + b; }`

Procedura

- **Definice:** Podprogram, který provede posloupnost příkazů, ale **nevrací žádnou hodnotu** explicitně.
- **Návratový typ:** V jazyce C se procedura realizuje jako funkce s návratovým typem `void`.
- **Typické použití:** Provádění akcí s **vedlejšími efekty**, jako je výpis na konzoli, zápis do souboru, modifikace globálních proměnných nebo změna stavu objektů předaných odkazem.
- **Příklad (C):** `c void vypisPozdrav(const char *jmeno) { printf("Ahoj, %s!\n", jmeno); }`

Metoda

- **Definice:** Funkce, která je přidružená k nějakému **objektu nebo třídě** v objektově orientovaném programování (OOP).
- **Kontext:** Metoda je volána na instanci objektu a má přístup k vnitřním datům (atributům) daného objektu prostřednictvím implicitního ukazatele/reference (`this` v C++, `self` v Pythonu, `this` v Javě).
- **Zapouzdření:** Metody jsou klíčové pro princip zapouzdření, kdy skrývají vnitřní implementaci objektu a vystavují pouze veřejné rozhraní.
- **Příklad (C++):** `cpp class Clovek { public: std::string jmeno; void pozdrav() { // Metoda std::cout << "Ahoj, jmenuji se " << this->jmeno << std::endl; } };`

Makra

- **Definice:** Nejsou to reálné funkce ani procedury. Jde o instrukce pro **preprocesor** (v C/C++ uvozené `#define`), které před samotnou kompilací mechanicky nahradí textový řetězec v kódu za definovanou hodnotu nebo výraz.
- **Textová substituce:** Preprocesor provede jednoduchou textovou substituci, aniž by rozuměl syntaxi jazyka nebo datovým typům.

- **Výhody:** Mohou zjednodušit opakovaný zápis kódu, umožňují podmíněnou kompilaci (`#ifdef`).
 - **Nevýhody (důležité pro SZZ):**
 - **Nekontrolují datové typy:** Nejsou typově bezpečné, což může vést k těžko odhalitelným chybám.
 - **Problémy s vyhodnocováním výrazů:** Mohou způsobit neočekávané výsledky kvůli chybějícím závorkám nebo vícenásobnému vyhodnocení argumentů.
 - **Obtížné ladění:** Chyby v makrech se obtížně ladí, protože se objevují až po expanzi preprocesorem.
 - **Zvětšování kódu (Code Bloat):** Při každém použití se kód makra vloží přímo, což může zvětšit velikost výsledného binárního souboru.
 - **Příklad (C) a úskalí:**

```
``c #define SQUARE(x) (x * x) // Chyba! Mělo by být ( (x) * (x) ) #define MAX(a, b) ((a) > (b) ? (a) : (b)) // Správné použití závorek
```



```
int main() { int a = 5; int b = SQUARE(a + 1); // Expanduje na (a + 1 * a + 1) = (5 + 5 + 1) = 11, ne 36! // Správně by mělo být: #define SQUARE(x) ((x) * (x)) // Pak by se expandovalo na ((a + 1) * (a + 1)) = (6 * 6) = 36 return 0; } ``
```

 - ****Alternativy:**** V C++ se pro typově bezpečné "makra" preferují inline funkce nebo šablony (`templates`).
-

B. Způsoby předávání parametrů

Když voláme podprogram (funkci, proceduru, metodu), musíme mu předat data, se kterými má pracovat. Rozlišujeme:

- **Formální parametry:** Proměnné definované v hlavičce podprogramu (např. `int a, int b` ve `int soucet(int a, int b)`).
- **Skutečné parametry (argumenty):** Reálné hodnoty nebo proměnné, které do podprogramu posíláme při volání (např. `soucet(5, 10)` – zde `5` a `10` jsou skutečné parametry).

Existují dva hlavní způsoby předávání parametrů, které ovlivňují, zda podprogram může změnit původní data:

1. Předávání hodnotou (Call by Value)

- **Princip:** Do formálního parametru podprogramu se **zkopíruje samotná hodnota** skutečného parametru. Podprogram si na zásobníku vytvoří vlastní lokální kopii této proměnné.
- **Důsledek:** Pokud podprogram uvnitř svého těla hodnotu tohoto parametru změní, mění se pouze jeho lokální kopie. **Původní proměnná ve volajícím programu zůstane nedotčena.**
- **Kdy použít:** Pro malé datové typy (int, char, float, bool) nebo když nechceme, aby podprogram měnil původní data.
- **V jazyce C:** Takto se standardně předávají všechny základní datové typy a struktury (pokud nejsou předány ukazatelem).

```
```c
```

## include

---

```
void zmena_hodnotou(int a) { printf("Uvnitr funkce (pred zmenou): a = %d\n", a); // Napr. 10
a = 99; // Změní pouze lokální kopii na zásobníku
printf("Uvnitr funkce (po zmene): a = %d\n", a); // Napr. 99 }
```

```
int main() { int mojeCislo = 10; printf("Pred volanim funkce: mojeCislo = %d\n",
mojeCislo); // 10
zmena_hodnotou(mojeCislo); printf("Po volani funkce: mojeCislo = %d\n",
mojeCislo); // Stále 10
return 0; } ```
```

## 2. Předávání odkazem / referencí (Call by Reference)

- **Princip:** Podprogramu se nepředává kopie hodnoty, ale **odkaz na původní paměťové místo (adresa)**, kde je proměnná uložena.
- **Důsledek:** Jakákoliv změna hodnoty parametru uvnitř podprogramu se **okamžitě projeví na původní proměnné** ve volajícím programu.
- **Kdy použít:**
  - Když chceme, aby podprogram modifikoval původní data.



- Pro předávání velkých datových struktur (např. velkých objektů, polí), aby se předešlo nákladnému kopírování a zvýšila se efektivita.
- Pro vrácení více hodnot z funkce (i když moderní jazyky nabízejí lepší alternativy jako tuple nebo struktury).

- **Realizace:**

- **V jazyce C++:** Existuje přímá podpora referencí pomocí operátoru `&` v deklaraci parametru.

```
```cpp
```

include

```
void zmena_odkazem_cpp(int &a) { // 'a' je reference na původní
proměnnou std::cout << "Uvnitř funkce (před změnou): a = " <<
a << std::endl; // Např. 10 a = 99; // Přepíše hodnotu na reálné
adrese původní proměnné std::cout << "Uvnitř funkce (po
změně): a = " << a << std::endl; // Např. 99 }
```

```
int main() { int mojeCislo = 10; std::cout << "Před voláním
funkce: mojeCislo = " << mojeCislo << std::endl; // 10
zmena_odkazem_cpp(mojeCislo); std::cout << "Po volání funkce:
mojeCislo = " << mojeCislo << std::endl; // 99 return 0; } ```
```

- **V jazyce C:** Předávání odkazem se simuluje **předáním ukazatele (pointeru) hodnotou**. Funkci předáme adresu proměnné (`&promenna`) a funkce s ní pracuje přes dereferenci (`*parametr`).

```
```c
```

## include

---

```
void zmena_odkazem_c(int a) { // 'a' je ukazatel na int
printf("Uvnitr funkce (pred zmenou): a = %d\n", a); // Napr. 10 a =
99; // Dereferencuje ukazatel a prepíše hodnotu na původní
adrese printf("Uvnitr funkce (po zmene): a = %d\n", a); // Napr.
99 }
```

```
int main() { int mojeCislo = 10; printf("Pred volanim funkce:
mojeCislo = %d\n", mojeCislo); // 10
zmena_odkazem_c(&mojeCislo); // Předáme adresu proměnné
printf("Po volani funkce: mojeCislo = %d\n", mojeCislo); // 99
return 0; } ``
```

- **\*\* const reference/ukazatele:\*\*** Pro efektivní předávání velkých objektů bez možnosti jejich modifikace se používají const reference (C++) nebo const` ukazatele (C). To kombinuje efektivitu předávání odkazem s bezpečností předávání hodnotou.

## Praktické použití předávání parametrů

- **Předávání hodnotou:**
  - Standardní pro primitivní datové typy (int, float, char).
  - Když je potřeba pracovat s kopií dat a neměnit originál.
- **Předávání odkazem/ukazatelem:**
  - Modifikace původních proměnných volajícího kódu.
  - Předávání velkých datových struktur (struktury, třídy, pole) pro optimalizaci výkonu (vyhnutí se kopírování).
  - Implementace funkcí, které "vrací" více hodnot (např. přes výstupní parametry).

## Nejčastější chyby u zkoušky (předávání parametrů)

- **Nepochopení rozdílu mezi hodnotou a odkazem:** Předpoklad, že změna parametru uvnitř funkce vždy ovlivní původní proměnnou.
- **Zapomenutí dereference ukazatele v C:** Předání adresy ( &var ) a následné použití var místo \*var uvnitř funkce.
- **Předávání velkých struktur hodnotou:** Způsobuje neefektivní kopírování a zpomalení programu.

- **Nezáměrná modifikace dat:** Předání odkazem, když se očekávalo, že data zůstanou nezměněna (řešením je `const` reference/ukazatel).
- 

## C. Globální a lokální proměnné (Rozsah platnosti a životní cyklus)

Proměnné se liší tím, kde v kódu jsou viditelné (**rozsah platnosti / scope**) a jak dlouho v paměti existují (**životní cyklus / lifetime**).

### 1. Lokální proměnné

- **Definice:** Jsou deklarovány **uvnitř těla funkce** nebo uvnitř libovolného bloku ohraničeného složenými závorkami `{ }` (např. uvnitř `if`, `for`, `while` cyklu).
- **Rozsah platnosti (Scope):** Jsou viditelné a přístupné **pouze uvnitř této funkce / bloku**. Ostatní funkce nebo bloky o nich nevědí a nemají k nim přístup.
- **Životní cyklus (Lifetime):** Vznikají na **zásobníku (Stack)** při vstupu do funkce/bloku a **automaticky zanikají** (paměť je uvolněna) při opuštění funkce/bloku. Pokud funkci zavoláte znovu, lokální proměnné začínají s novými (neinicializovanými, pokud nejsou explicitně inicializovány) hodnotami.
- **Příklad (C):** ````c #include`

```
void mojeFunkce() { int lokalniPromenna = 20; // Lokální proměnná pro mojeFunkce printf("Uvnitr mojeFunkce: lokalniPromenna = %d\n", lokalniPromenna); }
```

```
int main() { int dalsiLokalni = 5; // Lokální proměnná pro main // printf("%d", lokalniPromenna); // Chyba kompilace: lokalniPromenna není v tomto scope viditelná mojeFunkce(); printf("Uvnitr main: dalsiLokalni = %d\n", dalsiLokalni); return 0; } ```
```

### 2. Globální proměnné

- **Definice:** Jsou deklarovány **mimo jakoukoliv funkci**, obvykle na úplném začátku zdrojového souboru nebo v hlavičkovém souboru.
- **Rozsah platnosti (Scope):** Jsou viditelné a modifikovatelné v **celém programu** (pokud jsou deklarovány jako `extern` v jiných souborech) nebo v rámci celého souboru, kde jsou definovány (pokud jsou `static`).
- **Životní cyklus (Lifetime):** Vznikají při spuštění programu v **datovém segmentu paměti** (nebo BSS segmentu, pokud nejsou inicializovány) a existují po celou dobu jeho běhu, dokud program neskončí. Jsou inicializovány na nulu, pokud není uvedena explicitní inicializace.
- **Příklad (C):** ````c #include`

```
int globalniCislo = 100; // Globální proměnná
```

```
void zmenGlobalni() { globalniCislo = 200; // Může měnit globální
proměnnou printf("Uvnitr zmenGlobalni: globalniCislo = %d\n",
globalniCislo); }
```

```
int main() { printf("Pred volanim zmenGlobalni: globalniCislo = %d\n",
globalniCislo); // 100 zmenGlobalni(); printf("Po volani zmenGlobalni:
globalniCislo = %d\n", globalniCislo); // 200 return 0; } ```
```

- **static globální proměnné:** Pokud je globální proměnná deklarována jako `static`, její rozsah platnosti je omezen pouze na **jeden zdrojový soubor**, ve kterém je definována. Není viditelná z jiných souborů, i když jsou linkovány dohromady. To pomáhá omezit některé nevýhody globálních proměnných.

### **Varování pro státnice: Používání globálních proměnných**

Používání globálních proměnných se v moderním softwarovém inženýrství obecně považuje za **špatnou praxi (anti-pattern)** a je třeba se mu vyhybat, pokud to není absolutně nezbytné a odůvodněné. Důvody jsou následující:

- **Nečitelnost a obtížná údržba kódu:** Je těžké sledovat, které části programu globální proměnnou mění a kdy. To vede k "spaghetti kódu"

a skrytým závislostem.

- **Skryté závislosti (Side-effects):** Funkce, které modifikují globální proměnné, mají vedlejší efekty, které nejsou zřejmé z jejich hlavičky. To ztěžuje pochopení a testování funkcí.
- **Snížená znovupoužitelnost:** Funkce závislé na globálních proměnných jsou méně znovupoužitelné v jiných kontextech, protože vyžadují existenci těchto globálních proměnných.
- **Problémy v multithreadingovém prostředí (Race Conditions):** V prostředí s více vlákný (což je téma otázky č. 17) může souběžný přístup k globálním proměnným vést k **souběhům (Race Conditions)**, kdy výsledek závisí na náhodném pořadí vykonávání vláken. To vyžaduje složitou synchronizaci, která je náchylná k chybám (např. deadlocky).
- **Ztížené testování:** Testování funkcí s globálními proměnnými je obtížnější, protože je nutné nastavit a resetovat globální stav před každým testem.

Místo globálních proměnných se obvykle preferuje předávání dat jako parametrů nebo použití objektově orientovaných principů (zapouzdření dat do objektů a přístup k nim přes metody).

## Praktické použití globálních a lokálních proměnných

- **Lokální proměnné:**
  - Standardní pro většinu proměnných v programu.
  - Zajišťují izolaci kódu a minimalizují vedlejší efekty.
  - Automatická správa paměti (na zásobníku).
- **Globální proměnné (omezeně):**
  - Konfigurační nastavení, která se nemění za běhu programu (často se řeší jako `const` globální proměnné nebo konfigurační objekty).
  - Přístup k hardwarovým registrům v embedded systémech (kde je přímý přístup nutný).
  - V některých specifických architekturách (např. singletons, ale i ty se dají lépe řešit).

## Nejčastější chyby u zkoušky (globální/lokální proměnné)

- **Záměna rozsahu platnosti a životního cyklu:** Nepochopení, kdy proměnná vzniká a zaniká.
  - **Nadměrné používání globálních proměnných:** Ignorování rizik a nevýhod, zejména v kontextu multithreadingu.
  - **Předpoklad, že lokální proměnná si pamatuje hodnotu mezi voláními funkce:** Lokální proměnné jsou po opuštění funkce zničeny.
  - **Nepochopení inicializace:** Globální proměnné jsou implicitně inicializovány na nulu, lokální proměnné (bez `static` ) nejsou.
- 

## Typické zkuškové otázky

### 1. Vysvětlete rozdíl mezi funkcí, procedurou a metodou a uveďte příklady jejich použití.

#### Odpověď:

- **Funkce:** Blok kódu, který přijímá vstupy (parametry), provádí výpočet a **vrací hodnotu** jako svůj výsledek (pomocí `return` ). Je navržena pro výpočty a transformace dat bez vedlejších efektů.
  - *Příklad:* `int soucet(int a, int b) { return a + b; }`
- **Procedura:** Blok kódu, který provádí posloupnost příkazů, ale **nevrací explicitně žádnou hodnotu** (v C/C++ má návratový typ `void` ). Jejím hlavním účelem je provádět akce s vedlejšími efekty.
  - *Příklad:* `void vypisText(const char *text) { printf("%s\n", text); }`
- **Metoda:** Funkce, která je **přidružená k objektu nebo třídě** v objektově orientovaném programování. Je volána na instanci objektu a má přístup k jeho vnitřním datům (atributům) a může měnit jeho stav.
  - *Příklad (C++):* `class Auto { public: void nastartuj() { /* ... */ } };`

### 2. Popište způsoby předávání parametrů (hodnotou a odkazem/referencí) a vysvětlete, kdy který použít.

#### Odpověď:

- **Předávání hodnotou (Call by Value):** Funkci se předá **kopie hodnoty** skutečného parametru. Funkce pracuje s touto kopií na zásobníku, a proto jakékoli změny uvnitř funkce neovlivní původní proměnnou ve volajícím programu.
  - *Kdy použít:* Pro malé datové typy (int, char, float) a vždy, když nechceme, aby funkce měnila původní data.
- **Předávání odkazem / referencí (Call by Reference):** Funkci se předá **adresa paměťového místa** původní proměnné. Funkce pak pracuje přímo s původními daty, a proto jakékoli změny uvnitř funkce se projeví i ve volajícím programu. V C++ se používají reference ( & ), v C se simuluje předáním ukazatele ( \* ).
  - *Kdy použít:* Když chceme, aby funkce modifikovala původní data, nebo pro efektivní předávání velkých datových struktur (objektů, polí), aby se předešlo nákladnému kopírování.

### 3. Vysvětlete rozdíl mezi lokálními a globálními proměnnými z hlediska rozsahu platnosti a životního cyklu. Jaká jsou rizika používání globálních proměnných?

#### Odpověď:

- **Lokální proměnné:**
  - **Rozsah platnosti:** Viditelné a přístupné pouze uvnitř funkce nebo bloku kódu, kde jsou definovány.
  - **Životní cyklus:** Vznikají na zásobníku při vstupu do funkce/bloku a automaticky zanikají při jeho opuštění.
- **Globální proměnné:**
  - **Rozsah platnosti:** Viditelné a přístupné v celém programu (nebo v celém souboru, pokud jsou `static` ).
  - **Životní cyklus:** Vznikají v datovém segmentu paměti při spuštění programu a existují po celou dobu jeho běhu.
- **Rizika používání globálních proměnných:**
  - **Nečitelnost a obtížná údržba:** Je těžké sledovat, které části kódu globální proměnnou mění.
  - **Skryté závislosti (Side-effects):** Funkce mohou mít neočekávané vedlejší efekty, které nejsou zřejmé z jejich rozhraní.

- **Problémy v multithreadingu (Race Conditions):** Souběžný přístup k globálním proměnným z více vláken bez řádné synchronizace vede k nepředvídatelným výsledkům.
- **Snížená znovupoužitelnost:** Funkce jsou pevně svázány s globálním stavem.

## 4. Co je makro a jaké jsou jeho hlavní nevýhody oproti funkcím?

### Odpověď:

- **Makro:** Je to instrukce pro preprocesor, která provádí **textovou substituci** v kódu před samotnou kompilací. Není to skutečná funkce, ale jen náhrada textového řetězce.
  - **Hlavní nevýhody:**
    - **Není typově bezpečné:** Preprocesor nekontroluje datové typy, což může vést k chybám.
    - **Problémy s vyhodnocováním výrazů:** Může vést k neočekávaným výsledkům kvůli chybějícím závorkám nebo vícenásobnému vyhodnocení argumentů.
    - **Obtížné ladění:** Chyby v makrech se těžko detekují a ladí, protože se projevují až po expanzi.
    - **Zvětšování kódu (Code Bloat):** Kód makra se vloží na každé místo použití, což může zvětšit velikost binárního souboru.
- 

## Shrnutí kapitoly

Členění programu do menších jednotek, jako jsou funkce, procedury a metody, je základem pro psaní přehledného, udržitelného a znovupoužitelného kódu. Funkce se zaměřují na výpočty s návratovou hodnotou, procedury na akce s vedlejšími efekty a metody na chování objektů. Makra nabízejí textovou substituci, ale s rizikem typové nebezpečnosti a obtížného ladění. Způsob předávání parametrů (hodnotou vs. odkazem) určuje, zda podprogram může modifikovat původní data. Rozsah platnosti a životní cyklus proměnných (lokální na zásobníku, globální v datovém segmentu) jsou klíčové pro správu paměti a kontrolu přístupu.



Zatímco lokální proměnné jsou preferované pro svou izolaci a bezpečnost, globální proměnné by měly být používány s velkou opatrností kvůli rizikům, jako jsou skryté závislosti a problémy v multithreadingovém prostředí.

---

Níže je připravený studijní materiál k okruhu "15. Objektově orientované programování" pro státní závěrečné zkoušky, doplněný a naformátovaný dle zadaných pravidel.

---

# 15. Objektově orientované programování

---

## Klíčové pojmy

- třída
- objekt / instance
- atribut
- metoda
- zapouzdření
- dědičnost
- polymorfismus
- správa přístupu (modifikátory viditelnosti: `private` , `protected` , `public` )
- abstraktní třída
- rozhraní (interface)
- genericita
- kompozice
- přetěžování metod (method overloading)
- přepisování metod (method overriding)
- tabulka virtuálních metod (VMT)

## A. Význam OOP

Objektově orientované programování (OOP) vzniklo jako reakce na složitost velkých projektů psaných procedurálně. Zatímco v procedurálním

programování (např. v jazyce C) je program rozdělen na data a funkce, které s nimi manipulují, v OOP se data a související funkčnost spojují do jednoho celku zvaného objekt. Tento přístup vede k lepší modularitě, znovupoužitelnosti kódu a snadnější údržbě.

- **Třída (Class):** Je to obecný vzor, šablona nebo "předpis" (datový typ), který definuje, jaké vlastnosti (atributy) a chování (metody) budou mít objekty z ní vytvořené.
- **Objekt / Instance:** Je konkrétní, hmatatelný zástupce dané třídy alokovaný v paměti. Každý objekt má svůj vlastní stav (hodnoty atributů).
- **Atributy:** Proměnné uvnitř třídy reprezentující stav objektu.
- **Metody:** Funkce uvnitř třídy reprezentující chování objektu.

## B. Tři základní pilíře OOP a správa přístupu

### 1. Zapouzdření (Encapsulation)

- **Princip:** Sdružení dat (atributů) a metod, které s těmito daty pracují, do jednoho celku (třídy). Zároveň se skrývá vnitřní stav objektu před okolním světem.
- **Cíl:** Zamezit tomu, aby vnější kód mohl přímo a nekontrolovaně měnit vnitřní proměnné objektu, což by mohlo narušit jeho konzistenci. Tím se chrání integrita dat a zjednodušuje se údržba kódu, protože změny v interní implementaci třídy neovlivní vnější kód, pokud se nemění veřejné rozhraní.
- **Realizace:** Atributy se typicky označí jako soukromé ( `private` ) a přístup k nim se umožní pouze přes definované veřejné metody – tzv. Gettery (pro čtení) a Settery (pro zápis), ve kterých lze data validovat nebo provádět další logiku.

### Správa přístupu (Přístupová práva / Modifikátory viditelnosti)

- `private` : Člen (atribut nebo metoda) je přístupný pouze uvnitř samotné třídy, ve které je definován.

- **protected** : Člen je přístupný uvnitř třídy, ve které je definován, a ve všech jejích odvozených (podřízených) třídách.
- **public** : Člen je přístupný odkudkoliv z vnějšího kódu.

## 2. Dědičnost (Inheritance)

- **Princip:** Umožňuje vytvářet nové třídy (potomky / podtřídy) na základě tříd již existujících (rodič / nadtřída). Nová třída přebírá (dědí) atributy a metody třídy původní a může je rozšiřovat o nové, nebo upravovat ty stávající.
- **Cíl:** Znovupoužitelnost kódu (princip DRY – Don't Repeat Yourself) a vytváření logických hierarchií, které vyjadřují vztah "je prvkem" (tzv. "is-a" relationship), např. Pes je Zvíře .
- **Příklad:** Třída Auto může dědit od třídy Vozidlo . Auto zdědí obecné vlastnosti Vozidla (např. rychlost, počet kol) a přidá si specifické (např. počet dveří, typ motoru).

## 3. Polymorfismus (Mnohotvárnost)

- **Princip:** Schopnost různých objektů reagovat na stejné volání metody odlišným způsobem. Umožňuje přistupovat k objektům různých podtříd jednotným způsobem skrze rozhraní jejich společného rodiče nebo implementovaného rozhraní.
- **Cíl:** Zvýšení flexibility a rozšiřitelnosti kódu. Kód může pracovat s obecnými typy, aniž by musel znát konkrétní implementaci.

### Typy polymorfismu

- **Statický (při překladu – Compile-time Polymorphism):**
  - **Přetěžování metod (Method Overloading):** Více metod ve stejné třídě má stejné jméno, ale liší se počtem nebo typem parametrů. Kompilátor rozhoduje, která metoda se zavolá, na základě signatury volání.
  - **Příklad:**

```
java class Calculator { public int add(int a, int b) { return a + b; } public double add(double a, double b) { return a + b; } }
```
- **Dynamický (za běhu – Runtime Polymorphism):**

- **Přepisování metod (Method Overriding):** Potomek předefinuje metodu, kterou podědil od rodiče. O tom, která verze metody se skutečně zavolá, rozhoduje procesor až za běhu programu podle toho, jakého typu je reálný objekt v paměti (využívá se k tomu tzv. tabulka virtuálních metod – VMT).
- **Příklad:**

```
java class Animal { public void makeSound() { System.out.println("Animal makes a sound"); } } class Dog extends Animal { @Override public void makeSound() { System.out.println("Woof!"); } }
```

## Dědičnost vs. Kompozice

- **Dědičnost (Inheritance):** Vyjadřuje vztah "je prvkem" (is-a relationship).
  - **Výhody:** Znovupoužitelnost kódu, snadné rozšíření základního chování.
  - **Nevýhody:** Silné provázání (tight coupling) mezi rodičem a potomkem. Změny v rodičovské třídě mohou ovlivnit potomky (tzv. "fragile base class" problem). Omezení na jednu nadtřidu v mnoha jazycích (např. Java, C#).
- **Kompozice (Composition):** Vyjadřuje vztah "má část" (has-a relationship). Objekt obsahuje instance jiných objektů jako své atributy a deleguje jim část své funkčnosti.
  - **Výhody:** Volnější provázání (loose coupling), větší flexibilita (chování lze měnit za běhu výměnou komponent), snadnější testování.
  - **Nevýhody:** Vyžaduje více kódu pro delegování volání.
- **Kdy co použít:**
  - **Dědičnost** je vhodná, když existuje jasná hierarchie a vztah "je prvkem" je silný a neměnný (např. Kruh je Tvar).
  - **Kompozice** je obecně preferována pro větší flexibilitu a snížení provázanosti, zejména když vztah "má část" je proměnlivý nebo když objekt může mít různé varianty chování (např. Auto má Motor, ale motor může být vyměněn za jiný typ). Pravidlo "prefer composition over inheritance" je v OOP často doporučováno.

## C. Abstraktní třídy a Rozhraní (Interface)

Slouží k definování kontraktů v architektuře softwaru, čímž vynucují jednotný design aplikací.

### Abstraktní třída

- **Definition:** Třída, od které nelze vytvořit instanci (objekt). Slouží jako základ pro jiné třídy.
- **Obsah:** Může (ale nemusí) obsahovat tzv. abstraktní metody, což jsou metody bez implementace (mají jen hlavičku). Každý potomek, který z této třídy dědí, musí tyto abstraktní metody povinně naimplementovat. Může obsahovat i běžné implementované metody a členské atributy (včetně `private` a `protected`).
- **Účel:** Poskytuje částečnou implementaci a zároveň definuje rozhraní pro své potomky. Vztah "je prvkem" (is-a).

### Rozhraní (Interface)

- **Definition:** Představuje čistý funkční kontrakt. Říká, co objekt umí, ale neřeší, jak to dělá.
- **Obsah:** Tradičně obsahuje pouze hlavičky metod (bez jakékoliv implementace) a konstanty. V moderních jazycích (Java 8+, C# 8+) mohou rozhraní obsahovat i `default` implementace metod nebo statické metody.
- **Účel:** Definuje sadu chování, které musí třída implementovat. Třída rozhraní neoddědí, ale implementuje ho. Výhodou je, že v jazycích jako Java nebo C# sice nelze dědit z více tříd najednou (vícenásobná dědičnost), ale třída může implementovat libovolné množství rozhraní. Vztah "umí" (can-do).

## D. Genericita a její využití

- **Definition:** Genericita (šablony / Templates v C++, Generics v Javě/C#) umožňuje psát kód (třídy nebo funkce) nezávisle na konkrétním datovém typu, se kterým bude pracovat. Datový typ se stává

parametrem, který se specifikuje až při vytváření instance nebo volání funkce.

- **Cíl:** Zvýšení znovupoužitelnosti kódu a typové bezpečnosti.

## Využití genericity

- **Datové kontejnery (Kolekce):** Typickým příkladem jsou dynamická pole, spojové seznamy, fronty nebo mapy. Místo toho, abychom psali samostatný kód pro spojový seznam celých čísel ( `IntList` ), seznam desetinných čísel ( `FloatList` ) a seznam textových řetězců ( `StringList` ), napíšeme jednu generickou šablonu `List<T>` . Datový typ `T` se doplní až při inicializaci (např. `List<int>` nebo `List<String>` ).
- **Typová bezpečnost (Type Safety):** Kompilátor díky genericitě už v době překladu kontroluje, zda do kontejneru nevkládáme nesprávný datový typ. To eliminuje nutnost přetypování za běhu programu a předchází chybám typu `ClassCastException` (v Javě) nebo podobným chybám v jiných jazycích.
- **Algoritmy:** Umožňuje implementovat algoritmy (např. řazení, vyhledávání) tak, aby fungovaly s různými datovými typy, aniž by bylo nutné psát specifické verze pro každý typ.

## Praktické použití

- **GUI aplikace:** Většina moderních GUI frameworků (např. .NET WinForms/WPF, Java Swing/JavaFX) je postavena na OOP principech pro reprezentaci komponent (tlačítka, okna) a zpracování událostí.
- **Podnikové informační systémy:** Modelování reálných entit (zákazník, objednávka, produkt) jako objektů a jejich interakcí.
- **Modelování domény:** Vytváření objektových modelů, které přesně odrážejí koncepty a vztahy v dané problémové oblasti.
- **Knihovny a frameworky:** OOP je základem pro návrh většiny rozsáhlých softwarových knihoven a frameworků, což umožňuje jejich flexibilní použití a rozšiřitelnost.

## Nejčastější chyby u zkoušky

- **Používání dědičnosti pro pouhé sdílení kódu:** Dědičnost by měla být použita, když existuje silný vztah "is-a", nikoli jen pro znovupoužitelnost kódu, kde by byla vhodnější kompozice.
- **Veřejné atributy místo rozhraní:** Přímý přístup k atributům ( `public` atributy) porušuje zapouzdření a vede k nekontrolovaným změnám stavu objektu. Měly by se používat `private` atributy s `public` gettery a settery.
- **Zaměňování třídy a objektu:** Třída je šablona, objekt je konkrétní instance této šablony v paměti.

## Typické zkouškové otázky

### 1. Vysvětlete zapouzdření, dědičnost a polymorfismus.

- **Zapouzdření:** Princip sdružení dat (atributů) a metod do jednoho celku (třídy) a skrytí vnitřního stavu objektu před vnějším světem. Přístup k datům je řízen přes veřejné metody. Cílem je ochrana datové integrity a snížení komplexity.
- **Dědičnost:** Mechanismus, který umožňuje vytvářet nové třídy (potomky) na základě existujících tříd (rodičů). Potomek přebírá atributy a metody rodiče a může je rozšiřovat nebo přepisovat. Vyjadřuje vztah "je prvkem" (is-a). Cílem je znovupoužitelnost kódu a tvorba hierarchií.
- **Polymorfismus:** Schopnost různých objektů reagovat na stejné volání metody odlišným způsobem. Umožňuje pracovat s objekty různých typů jednotným rozhraním. Existuje statický (přetěžování metod) a dynamický (přepisování metod) polymorfismus. Cílem je flexibilita a rozšiřitelnost.

### 2. Co je rozhraní?

- Rozhraní (interface) je kontrakt, který definuje sadu metod, které musí třída implementovat. Tradičně obsahuje pouze hlavičky metod bez implementace a konstanty. Třída rozhraní



implementuje, nikoli dědí. Slouží k vynucení určitého chování a umožňuje vícenásobnou "dědičnost" chování tam, kde jazyk nepodporuje vícenásobnou dědičnost tříd.

### 3. Kdy dát přednost kompozici?

- Kompozici je vhodné dát přednost, když vztah mezi objekty je typu "má část" (has-a relationship) a vyžaduje se vysoká flexibilita a nízká provázanost (loose coupling). Umožňuje snadnější změnu chování objektu za běhu (výměnou komponenty) a předchází problémům spojeným se silným provázáním dědičnosti, jako je "fragile base class" problem.

## Shrnutí kapitoly

OOP organizuje program kolem objektů s odpovědnostmi. Dobré OOP omezuje vazby a vystavuje stabilní rozhraní. Základními pilíři jsou zapouzdření, dědičnost a polymorfismus, které společně s abstraktními třídami, rozhraními a genericitou umožňují vytvářet modulární, znovupoužitelné a snadno udržitelné softwarové systémy.

---

Zde je dokonalý studijní materiál k okruhu "16. Tvorba aplikací v prostředí konzole a MS Windows. Vývojová prostředí." připravený pro státní závěrečné zkoušky.

---

# Kapitola 16: Tvorba aplikací v prostředí konzole a MS Windows. Vývojová prostředí.

---

## Klíčové pojmy

- konzolová aplikace (CLI)
  - GUI aplikace (Graphical User Interface)
  - standardní vstup (stdin), výstup (stdout), chybový výstup (stderr)
  - událostmi řízené programování (Event-Driven Programming)
  - smyčka zpráv (Message Loop)
  - zpráva (Message)
  - procedura okna (Window Procedure / WndProc)
  - Win32 API
  - frameworky (MFC, Qt, wxWidgets, Windows Forms, WPF)
  - XAML (Extensible Application Markup Language)
  - vývojové prostředí (IDE - Integrated Development Environment)
  - textový editor (zvýrazňování syntaxe, automatické doplňování, refaktorování)
  - kompilátor, linker, build systém
  - debugger (breakpoints, krokování, sledování proměnných)
  - profiler
  - verzovací systém
-

## 16.1 Tvorba aplikací v prostředí konzole (CLI)

Konzolové aplikace (Command Line Interface) běží v textovém režimu a komunikují s uživatelem prostřednictvím textového vstupu a výstupu. Jejich tok řízení je striktně sekvenční a lineární.

### Princip fungování

Program se spustí, lineárně vykonává instrukce, v případě potřeby zastaví vykonávání a počká na textový vstup od uživatele, zpracuje ho, vypíše výstup a po dosažení konce funkce `main()` (nebo ekvivalentní vstupní funkce) skončí.

### Vstup a výstup

Využívají se standardní datové proudy (streams), které jsou automaticky dostupné pro každou konzolovou aplikaci:

- **`stdin` (standardní vstup):** Typicky klávesnice, ze které program čte uživatelský vstup. V jazyce C se používají funkce jako `scanf()` nebo `getchar()`.
- **`stdout` (standardní výstup):** Typicky textová obrazovka (konzole), kam program vypisuje běžné výsledky a informace. V jazyce C se používá `printf()`.
- **`stderr` (standardní chybový výstup):** Vyhrazený pro chybová hlášení a diagnostické zprávy. Je oddělen od `stdout`, což umožňuje přesměrování chybových zpráv jinam než běžného výstupu. V jazyce C se používá `fprintf(stderr, ...)`.

Standardní proudy lze přesměrovat (např. `program < input.txt > output.txt 2> errors.log`) nebo propojit pomocí `pipe` (pipes) pro řetězení programů (např. `program1 | program2`).

### Výhody

- **Minimální režie:** Nízké nároky na systémové prostředky a rychlý start.

- **Snadná automatizace a skriptování:** Ideální pro systémové utility, dávkové zpracování a skripty.
- **Jednodušší debugování:** Méně komplexní stav, lineární tok usnadňuje sledování chyb.
- **Zaměření na algoritmus:** Programátor se soustředí čistě na logiku a algoritmus, nikoliv na vzhled.
- **Multiplatformnost:** Často snadno přenositelné mezi různými operačními systémy.

## Nevýhody

- **Omezená interakce:** Pouze textová komunikace, nevhodné pro vizuálně bohaté úlohy.
  - **Není intuitivní pro běžné uživatele:** Vyžaduje znalost příkazového řádku.
- 

## 16.2 Tvorba aplikací v prostředí MS Windows (GUI)

Aplikace s grafickým uživatelským rozhraním (Graphical User Interface) pro Windows fungují na zcela odlišném paradigmatu: **událostmi řízeném programování (Event-Driven Programming)**. Program neběží lineárně, ale "sedí" v paměti a čeká, až uživatel nebo systém něco udělá.

### 16.2.1 Architektura Win32 API a smyčka zpráv (Message Loop)

Základním stavebním kamenem každé nativní Windows aplikace je registrace okna a spuštění nekonečné smyčky, která zachytává systémové zprávy.

- **Zpráva (Message):** Jakákoliv akce (stisk klávesy, pohyb myši, kliknutí na tlačítko, požadavek na překreslení okna, událost časovače) je operačním systémem přetvořena do struktury zprávy ( MSG ) a vložena

do fronty zpráv dané aplikace. Zprávy mají identifikátory jako

`WM_PAINT` , `WM_LBUTTONDOWN` , `WM_DESTROY` .

- **Smyčka zpráv (Message Loop):** Jádro GUI aplikace. Jedná se o nekonečnou smyčku, která neustále vybírá zprávy z fronty a předává je k dalšímu zpracování.
  - `GetMessage()` : Blokující funkce, která čeká na novou zprávu ve frontě.
  - `PeekMessage()` : Neblokující funkce, která zkontroluje frontu zpráv. Pokud zpráva je, zpracuje ji; pokud ne, vrátí se a program může vykonávat jiné úlohy (např. vykreslování animace).
  - `TranslateMessage()` : Překládá zprávy o stisku virtuálních kláves na zprávy o znakových vstupech ( `WM_CHAR` ).
  - `DispatchMessage()` : Předá zprávu příslušné proceduře okna (Window Procedure).
- **Procedura okna (Window Procedure / WndProc):** Callback funkce definovaná vývojářem, která je volána operačním systémem pro zpracování konkrétních zpráv. Obsahuje typicky velkou konstrukci `switch-case` , která reaguje na různé typy zpráv (např. `WM_PAINT` pro vykreslení obsahu okna, `WM_LBUTTONDOWN` pro kliknutí levým tlačítkem myši, `WM_DESTROY` pro zavření okna).
- **Registrace a vytvoření okna:** Před spuštěním smyčky zpráv je nutné definovat třídu okna ( `WNDCLASSEX` ) a zaregistrovat ji pomocí `RegisterClassEx()` . Následně se vytvoří samotné okno pomocí `CreateWindowEx()` .

## 16.2.2 Moderní frameworky pro Windows

Psát aplikace přímo ve Win32 API v C/C++ je extrémně zdlouhavé a náchylné k chybám. Proto se v praxi používají objektové nadstavby a frameworky, které zjednodušují vývoj a poskytují vyšší úroveň abstrakce.

- **C++ (Nativní/Multiplatformní):**
  - **MFC (Microsoft Foundation Classes):** Starší objektový obal nad Win32 API. Zjednodušuje práci s okny, ovládacími prvky a zprávami. Stále se používá v legacy systémech.

- **Qt:** Rozsáhlý multiplatformní framework pro vývoj GUI aplikací, ale i síťových, databázových a multimediálních aplikací. Používá vlastní mechanismus signálů a slotů pro zpracování událostí. Je oblíbený pro desktopové aplikace, embedded systémy, CAD software a hry.
- **wxWidgets:** Další multiplatformní framework, který se snaží o nativní vzhled a chování na každém operačním systému tím, že používá nativní GUI prvky daného OS.
- **C# a .NET Ecosystem (Standard pro Windows):**
  - **Windows Forms (WinForms):** Starší, ale stále velmi používaný framework pro rychlý vývoj desktopových aplikací. Využívá pixelové pozicování prvků a přímé provázání s událostmi tlačítek a dalších ovládacích prvků. Je založen na událostním modelu.
  - **WPF (Windows Presentation Foundation):** Modernější přístup k vývoji GUI, který striktně odděluje vzhled od logiky. Vzhled se definuje pomocí deklarativního jazyka XAML (Extensible Application Markup Language, podobný XML) a podporuje pokročilé datové vazby (Data Binding), styly, šablony a hardwarovou akceleraci grafiky přes DirectX. Je vhodný pro bohatá a vizuálně komplexní uživatelská rozhraní.

## Výhody GUI

- **Intuitivní interakce:** Vizuální prvky (tlačítka, menu, okna) jsou pro uživatele přirozenější.
- **Bohaté možnosti prezentace dat:** Grafy, obrázky, komplexní rozvržení.
- **Vhodné pro komplexní uživatelské úlohy:** Aplikace jako editory, grafické programy, kancelářské balíky.

## Nevýhody GUI

- **Vyšší režie:** Náročnější na systémové prostředky.
- **Složitější vývoj a debugování:** Událostmi řízený tok je komplexnější.
- **Závislost na OS:** Nativní GUI aplikace jsou často vázány na konkrétní operační systém.

- **Potřeba asynchronního programování:** Dlouhé operace musí běžet v samostatných vláknech, aby hlavní UI vlákno nezamrzlo.
- 

## 16.3 Vývojové prostředí (IDE - Integrated Development Environment)

Vývojové prostředí (IDE) je komplexní softwarový balík, který integruje všechny nástroje potřebné pro efektivní tvorbu aplikací do jednoho rozhraní. Cílem je zefektivnit celý životní cyklus vývoje softwaru.

### Základní součásti každého IDE

- **Chytrý textový editor:**
  - **Zvýrazňování syntaxe:** Barevné odlišení klíčových slov, komentářů, řetězců pro lepší čitelnost.
  - **Automatické doplňování kódu (IntelliSense/Autocompletion):** Navrhuje názvy proměnných, funkcí, tříd a metod během psaní kódu, což zrychluje psaní a snižuje chyby.
  - **Refaktorování:** Automatizované nástroje pro bezpečné změny struktury kódu (např. přejmenování proměnných, extrakce metod, přesun tříd).
  - **Navigace v kódu:** Rychlé přecházení na definice symbolů, hledání všech referencí, zobrazení hierarchie tříd.
  - **Formátování kódu:** Automatické uspořádání kódu podle předdefinovaných stylů.
- **Kompilátor / Linker (Sestavovací nástroje):**
  - **Kompilátor:** Překládá zdrojový kód z vyššího programovacího jazyka na strojový kód nebo mezikód.
  - **Linker:** Spojuje přeložené objektové soubory a potřebné knihovny do spustitelného programu.
  - **Build systém:** Spravuje proces sestavení projektu, včetně závislostí mezi soubory a knihovnami (např. Make, CMake, MSBuild, Gradle). Umožňuje sestavit projekt stiskem jednoho tlačítka.

- **Debugger (Ladicí nástroj):** Umožňuje spouštět kód v kontrolovaném režimu pro hledání a opravu chyb.
  - **Breakpoints (Body zastavení):** Místa v kódu, kde se vykonávání programu automaticky pozastaví.
  - **Krokování (Step-by-step execution):**
    - **Step Over :** Vykoná aktuální řádek kódu a zastaví se na dalším, aniž by vstoupil do volaných funkcí.
    - **Step Into :** Vstoupí do volané funkce, pokud je to možné.
    - **Step Out :** Dokončí vykonávání aktuální funkce a vrátí se k volajícímu.
  - **Sledování proměnných (Variable Watch):** Zobrazení aktuálních hodnot proměnných v paměti.
  - **Zásobník volání (Call Stack):** Zobrazuje posloupnost funkcí, které vedly k aktuálnímu bodu vykonávání.
  - **Zobrazení paměti (Memory View):** Umožňuje prohlížet obsah paměti.
- **Další integrované nástroje:**
  - **Profiler:** Analyzuje výkon programu (spotřebu CPU, paměti, I/O operace) pro identifikaci úzkých hrdel.
  - **Integrace s verzovacími systémy:** Podpora pro Git, SVN a další systémy pro správu verzí kódu.
  - **Vizuální návrháře GUI:** Nástroje pro drag-and-drop tvorbu uživatelského rozhraní.
  - **Nástroje pro testování:** Integrace s frameworky pro jednotkové a integrační testy.

## Příklad relevantních IDE

- **Microsoft Visual Studio:** Profesionální a robustní IDE pro Windows, špička pro vývoj v C++, C#, .NET a webových technologiích. Obsahuje vizuální návrháře oken, pokročilé profilační nástroje a bohatý ekosystém.
- **JetBrains IDEs (CLion, Rider, IntelliJ IDEA, PyCharm):** Moderní cross-platformní IDE, známé pro pokročilou statickou analýzu kódu,



refaktorování a inteligentní doplňování kódu. CLion je pro C++, Rider pro .NET/C#, IntelliJ IDEA pro Javu, PyCharm pro Python.

- **Visual Studio Code (VS Code):** Lehký, open-source a vysoce rozšiřitelný textový editor. Pomocí pluginů se dá proměnit v plnohodnotné vývojové prostředí pro širokou škálu jazyků a technologií (Python, JavaScript, TypeScript, C++, Java, Go, PHP, Assembler).
- 

## Typické zkuškové otázky

1. **Vysvětlete rozdíl mezi konzolovou a GUI aplikací z pohledu interakce s uživatelem a toku řízení.**

**Odpověď:**

- **Konzolová aplikace (CLI):** Interakce probíhá textově přes standardní vstup/výstup. Tok řízení je **sekvenční a lineární**; program se spouští, vykonává instrukce v daném pořadí, čeká na vstup a po dokončení obvykle končí.
- **GUI aplikace:** Interakce probíhá vizuálně pomocí grafických prvků (okna, tlačítka, menu). Tok řízení je **událostmi řízený (Event-Driven)**; program "sedí" v paměti a čeká na události (kliknutí myši, stisk klávesy, systémové zprávy) a reaguje na ně prostřednictvím callback funkcí.

2. **Popište princip událostmi řízeného programování a roli smyčky zpráv (message loop) v GUI aplikacích pro Windows.**

**Odpověď:**

- **Událostmi řízené programování:** Je paradigma, kde program reaguje na události generované uživatelem (např. kliknutí, stisk klávesy) nebo systémem (např. překreslení okna, časovač). Místo lineárního vykonávání kódu program čeká na události a spouští příslušné obslužné rutiny.
- **Smyčka zpráv (Message Loop):** Je základním mechanismem GUI aplikací ve Windows. Jedná se o nekonečnou smyčku, která neustále:
  1. **Vybírá zprávy** z fronty zpráv aplikace (pomocí `GetMessage()` nebo `PeekMessage()`).
  2. Případně **překládá zprávy** (např.

TranslateMessage() pro klávesové zkratky). 3. **Předává zprávy** k dalšímu zpracování příslušné proceduře okna ( DispatchMessage() ), která obsahuje logiku pro reakci na konkrétní typy zpráv.

### 3. Jaké jsou hlavní komponenty moderního vývojového prostředí (IDE) a k čemu slouží debugger?

**Odpověď:** Hlavní komponenty moderního IDE jsou:

- **Chytrý textový editor:** Pro psaní kódu s funkcemi jako zvýrazňování syntaxe, automatické doplňování, refaktorování a navigace.
- **Kompilátor / Linker (Sestavovací nástroje):** Pro překlad zdrojového kódu na spustitelný program a správu závislostí projektu.
- **Debugger (Ladicí nástroj):** Pro hledání a opravu chyb v programu.

**Debugger** slouží k:

- **Spouštění programu v kontrolovaném režimu:** Umožňuje pozastavit vykonávání programu v libovolném bodě.
- **Nastavování breakpointů:** Zastaví vykonávání na konkrétním řádku kódu.
- **Krokování kódu:** Vykonávání programu po jednotlivých řádcích ( Step Over , Step Into , Step Out ).
- **Sledování hodnot proměnných:** Zobrazení a změna hodnot proměnných v reálném čase.
- **Analýza zásobníku volání:** Zobrazení posloupnosti funkcí, které vedly k aktuálnímu bodu.

### 4. Porovnejte frameworky Windows Forms a WPF pro vývoj GUI aplikací v .NET.

**Odpověď:**

- **Windows Forms (WinForms):**
  - **Starší:** Založen na GDI+, používá nativní ovládací prvky Windows.
  - **Návrh:** Vizuální návrhář s pixelovým pozicováním prvků.
  - **Oddělení vzhledu a logiky:** Slabší, vzhled a logika jsou často těsně provázány v kódu.
  - **Výkon:** Méně efektivní pro komplexní grafiku, nepoužívá hardwarovou akceleraci.
  - **Použití:** Rychlý vývoj jednodušších desktopových aplikací, legacy

systémy.

- **WPF (Windows Presentation Foundation):**

- **Modernější:** Založen na DirectX, používá vektorovou grafiku.
- **Návrh:** Deklarativní pomocí XAML (Extensible Application Markup Language).
- **Oddělení vzhledu a logiky:** Silné, XAML pro vzhled, C# pro logiku (MVVM pattern). Podporuje Data Binding.
- **Výkon:** Využívá hardwarovou akceleraci, vhodný pro bohatá a vizuálně komplexní rozhraní.
- **Použití:** Moderní desktopové aplikace s bohatým UI, multimediální aplikace.

## 5. Uveďte výhody a nevýhody konzolových aplikací.

### Odpověď: Výhody:

- **Nízká reže:** Malé nároky na systémové prostředky.
- **Rychlý start:** Spouštějí se okamžitě.
- **Snadná automatizace a skriptování:** Ideální pro dávkové zpracování, systémové utility a skripty.
- **Jednodušší debugování:** Lineární tok řízení usnadňuje sledování chyb.
- **Multiplatformnost:** Často snadno přenositelné mezi OS.
- **Zaměření na logiku:** Vývojář se soustředí na algoritmus, ne na UI.

### Nevýhody:

- **Omezená interakce:** Pouze textová komunikace.
- **Není intuitivní pro běžné uživatele:** Vyžaduje znalost příkazového řádku.
- **Nevhodné pro vizuální úlohy:** Nelze zobrazovat grafiku, obrázky, komplexní rozvržení.

---

## Shrnutí kapitoly

Tato kapitola rozlišuje mezi dvěma základními typy aplikací: **konzolovými (CLI)** a **grafickými (GUI)**. Konzolové aplikace jsou charakteristické svým sekvenčním tokem řízení a textovou interakcí přes standardní proudy, což je

činí ideálními pro skriptování a systémové utility s nízkou režii. GUI aplikace, typické pro MS Windows, naopak využívají **událostmi řízené programování** a **smyčku zpráv**, která zpracovává uživatelské a systémové události. Pro zjednodušení vývoje GUI se používají moderní frameworky jako WPF (s deklarativním XAML) nebo Qt. Klíčovým nástrojem pro efektivní vývoj je **vývojové prostředí (IDE)**, které integruje editor, kompilátor a především **debugger** pro efektivní hledání a opravu chyb. Pochopení těchto paradigmat a nástrojů je zásadní pro tvorbu robustních a uživatelsky přívětivých aplikací.

---

Níže je připravený studijní materiál k okruhu 17, doplněný a naformátovaný dle vašich přísných pravidel.

---

## 17. Programátorské rozhraní operačního systému, vlákna, synchronizace

---

### Klíčové pojmy

- systémové volání (syscall)
- API (Application Programming Interface)
- uživatelský režim (user space)
- režim jádra (kernel space)
- zpracování zpráv (message processing)
- fronta zpráv (message queue)
- událostní smyčka (event loop)
- proces
- vlákno (thread)
- kontext procesu/vlákna
- multithreading
- sdílená paměť
- zásobník (stack)
- ukazatel instrukcí (Instruction Pointer)
- race condition (souběh)
- kritická sekce
- synchronizace
- mutex (mutual exclusion)
- semafor (binary, counting)
- podmínková proměnná (condition variable)
- deadlock (uváznutí)

- starvation (hladovění)
  - atomické operace
  - paměťové bariéry
- 

## A. Programátorská rozhraní operačního systému (API / Syscalls)

Aplikace běžící v **uživatelském prostoru (User Space)** nemají z bezpečnostních důvodů a pro zachování stability systému přímý přístup k hardwaru (disky, paměť, síťové karty) ani k privilegovaným operacím. Pokud chtějí provést jakoukoliv privilegovanou operaci (např. otevřít soubor, alokovat paměť, vytvořit vlákno, komunikovat po síti), musí o to požádat **jádro operačního systému (Kernel)**.

K tomu slouží **programátorské rozhraní (API)**, které interně volá **systémová volání (System Calls / Syscalls)**. Systémové volání je mechanismus, kterým program v uživatelském režimu požádá jádro operačního systému o provedení služby. Tento proces zahrnuje **přepnutí režimu (mode switch)** z uživatelského režimu do režimu jádra, často pomocí speciální instrukce (např. `INT 0x80` na x86 Linuxu nebo `syscall` instrukce). Jádro ověří oprávnění, provede požadovanou operaci a vrátí výsledek zpět do uživatelského režimu.

### Účel systémových volání:

- **Bezpečnost:** Izolace aplikací od hardwaru a od sebe navzájem.
- **Abstrakce:** Poskytují jednotné a stabilní rozhraní pro přístup k hardwaru, nezávislé na konkrétní implementaci.
- **Správa zdrojů:** Umožňují jádru efektivně spravovat a alokovat systémové zdroje mezi procesy.

### Příklady API a systémových volání:

- **POSIX (Portable Operating System Interface):** Standardizované rozhraní pro Unixové systémy (Linux, macOS). Definuje funkce jako:
  - `fork()` pro tvorbu procesů.

- `open()` , `read()` , `write()` , `close()` pro práci se soubory.
- `pthread_create()` pro tvorbu vláken.
- **Win32 API / Windows API:** Rozhraní specifické pro operační systémy MS Windows. Využívá funkce jako:
  - `CreateProcess()` pro tvorbu procesů.
  - `CreateThread()` pro tvorbu vláken.
  - `ReadFile()` , `WriteFile()` pro práci se soubory.

#### Kategorie systémových volání:

- **Správa procesů:** `fork()` , `exec()` , `wait()` , `exit()` , `CreateProcess()` .
  - **Správa souborů:** `open()` , `read()` , `write()` , `close()` , `CreateFile()` .
  - **Správa zařízení:** `ioctl()` .
  - **Správa informací:** `getpid()` , `time()` .
  - **Komunikace:** `pipe()` , `shmget()` , `socket()` .
- 

## B. Zpracování zpráv (Message Processing / Event-Driven)

Zpracování zpráv je základní technikou pro asynchronní komunikaci a **programování řízené událostmi (Event-Driven Programming)**. Je klíčové pro GUI aplikace, síťovou komunikaci a distribuované systémy.

**Princip:** Namísto přímého volání funkcí mezi komponentami generuje odesílatel **zprávy** (struktury obsahující ID události a data) a vkládá je do **fronty zpráv (Message Queue)**. Příjemce (např. hlavní vlákno aplikace) v nekonečné smyčce, nazývané **událostní smyčka (Event Loop)**, zprávy z fronty vybírá a zpracovává je pomocí dispečera (často s využitím **callback funkcí** nebo obslužných rutin událostí).

#### Fáze událostní smyčky:

1. **Čekání na událost:** Smyčka čeká na novou zprávu ve frontě.
2. **Vybrání události:** Jakmile je zpráva k dispozici, je vybrána z fronty.

3. **Zpracování události:** Zpráva je předána příslušné obslužné rutině (callbacku), která provede odpovídající akci.
4. **Opakování:** Smyčka se vrací k čekání na další událost.

#### Využití:

- **GUI aplikace (např. Windows Message Loop):** Uživatelské interakce (kliknutí myší, stisknutí klávesy), systémové události (změna velikosti okna, časovače) jsou transformovány na zprávy a vloženy do fronty.
  - **Asynchronní programování:** V moderních prostředích (např. Node.js, Python's asyncio) se událostní smyčka používá k efektivnímu zpracování I/O operací bez blokování hlavního vlákna.
  - **Mikroslužby a distribuované systémy:** Pro spolehlivé, neblokující propojení nezávislých procesů se používají specializované knihovny a message brokeři (např. ZeroMQ, RabbitMQ, Apache Kafka).
- 

## C. Programování vláken (Multithreading)

Abychom pochopili vlákna, musíme je nejprve odlišit od procesů:

- **Proces:** Spuštěná instance programu, která má vlastní izolovaný paměťový prostor přidělený operačním systémem. Procesy jsou od sebe striktně oddělené a komunikují pomocí mechanismů **meziprocesové komunikace (IPC - Inter-Process Communication)**, jako jsou roury, sdílená paměť nebo zprávy. Každý proces má svůj vlastní **kontext**, který zahrnuje stav registrů, paměťové mapy, otevřené soubory atd.
- **Vlákno (Thread):** Je nejmenší jednotka vykonávání kódu v rámci procesu. Jeden proces může mít více vláken. Všechna vlákna jednoho procesu **sdílejí stejný paměťový prostor** (haldy, globální proměnné, otevřené soubory, kód programu). Každé vlákno má však svůj vlastní **zásobník (Stack)** pro lokální proměnné, vlastní **ukazatel instrukcí (Instruction Pointer - IP)** a sadu registrů.

#### Typy vláken:



- **Vlákná na úrovni uživatele (User-Level Threads):** Spravována knihovnou v uživatelském prostoru, jádro o nich neví. Rychlé přepínání, ale jedno blokující volání blokuje celý proces.
- **Vlákná na úrovni jádra (Kernel-Level Threads):** Spravována jádrem operačního systému. Jádro může plánovat jednotlivá vlákna. Blokující volání jednoho vlákna neblokuje celý proces. Většina moderních OS používá tento model.

#### Stavy vlákna:

- **Nový (New):** Vlákno je vytvořeno, ale ještě nespustilo kód.
- **Připravený (Ready):** Vlákno čeká na přidělení procesoru plánovačem.
- **Běžící (Running):** Vlákno právě vykonává kód na procesoru.
- **Blokovaný/Čekající (Blocked/Waiting):** Vlákno čeká na nějakou událost (např. I/O operaci, uvolnění zámku, signál).
- **Ukončený (Terminated):** Vlákno dokončilo své vykonávání.

#### Výhody multithreadingu:

- **Rychlejší přepínání kontextu (Context Switch):** Přepínání mezi vlákny v rámci jednoho procesu je obvykle rychlejší než přepínání mezi procesy, protože není potřeba měnit paměťové mapy.
- **Snadné sdílení dat:** Vlákna sdílejí paměťový prostor, což usnadňuje komunikaci a sdílení dat bez složitých IPC mechanismů.
- **Paralelismus:** Možnost plně využít více jader moderních CPU k paralelnímu běhu úloh, což vede ke zvýšení výkonu.
- **Odezva:** Aplikace může zůstat responzivní (např. GUI) i během provádění dlouhých operací na pozadí.

#### Nevýhody multithreadingu:

- **Složitost programování:** Právě kvůli sdílené paměti je multithreading extrémně náchylný k chybám. Pokud dvě vlákna zapisují do stejného místa v paměti bez koordinace, program se začne chovat nepředvídatelně.
- **Race Conditions:** Riziko vzniku souběhů dat.
- **Deadlock a Starvation:** Riziko uvážnutí a hladovění.

- **Ladění:** Ladění souběžných programů je mnohem obtížnější než ladění sekvenčních programů.
- 

## D. Synchronizace (Vláken a Procesů)

Když více vláken nebo procesů přistupuje ke **sdílenému zdroji** (např. globální proměnné, souboru, hardwarovému zařízení) a alespoň jedno z nich provádí zápis, vzniká **souběh (Race Condition)**. Výsledek operace se stává závislým na náhodném proložení (interleaving) instrukcí jednotlivých vláken. Část kódu, která pracuje se sdíleným zdrojem a u které musíme zajistit, aby ji v jeden okamžik vykonávalo maximálně jedno vlákno, se nazývá **kritická sekce**.

K ochraně kritických sekcí a synchronizaci se používají tyto základní primitivy:

### 1. Mutex (Mutual Exclusion – Vzájemné vyloučení)

**Princip:** Funguje jako digitální "zámek". Vlákno, které chce vstoupit do kritické sekce, se pokusí mutex zamknout ( `lock()` nebo `acquire()` ). Pokud je mutex volný, vlákno ho uzamkne, provede kód v kritické sekci a při odchodu ho odemkne ( `unlock()` nebo `release()` ). Pokud je mutex již zamčený jiným vláknem, aktuální vlákno je operačním systémem uspáno (zablokováno) a čeká, dokud se zámek neuvolní. Mutex zajišťuje, že v daný okamžik může kritickou sekci vykonávat maximálně jedno vlákno.

**Příklad použití (pseudokód):**

```
lock(mutex);
// Kritická sekce: přístup ke sdíleným datům
shared_counter++;
unlock(mutex);
```

### 2. Semafor

**Princip:** Zobecněný mutex, který obsahuje čítač. Udává, kolik vláken může současně přistupovat ke zdroji. Operace se semaforem jsou `wait()` (nebo `P()`, `acquire()`) a `signal()` (nebo `V()`, `release()`).

- `wait()` : Dekrementuje čítač. Pokud je čítač záporný, vlákno se zablokuje.
- `signal()` : Inkrementuje čítač. Pokud jsou nějaká vlákna zablokovaná, jedno z nich probudí.

### Typy semaforů:

- **Binární semafor:** Má hodnotu 0 nebo 1. Chová se v podstatě jako mutex, zajišťuje vzájemné vyloučení.
- **Čítací semafor:** Používá se např. pro omezení přístupu k fondu připojení (Connection Pool) nebo k omezenému počtu zdrojů. Pokud je kapacita 5, semafor pustí dovnitř 5 vláken. Šesté vlákno se zablokuje a čeká, až některé z prvních pěti odejde a čítač se uvolní.

## 3. Podmínkové proměnné (Condition Variables)

**Princip:** Slouží k signalizaci mezi vlákny, která čekají na splnění určité podmínky. Podmínkové proměnné se vždy používají ve spojení s mutexem. Vlákno se může uspat a čekat na splnění určité podmínky ( `wait()` ). Jiné vlákno, které tuto podmínku splní, pošle signál ( `signal()` probudí jedno čekající vlákno, `broadcast()` probudí všechna čekající vlákna), který spící vlákno probudí.

### Příklad použití (Producent-Konzument problém):

- Vlákno-producent vloží data do bufferu a probudí spící vlákno-konzumenta, aby data zpracovalo.
- Vlákno-konzument čeká na data v bufferu. Pokud buffer prázdný, uspí se na podmínkové proměnné.

## Další synchronizační mechanismy:

- **Atomické operace:** Jednoduché operace (např. inkrementace, dekrementace, výměna hodnoty), které jsou garantovány hardwarem jako nedělitelné. Jsou rychlejší než mutexy pro jednoduché operace.
- **Paměťové bariéry (Memory Barriers):** Instrukce, které zajišťují, že operace s pamětí jsou provedeny v určitém pořadí, což je důležité pro

správnou viditelnost změn mezi procesorovými jádry.

- **Spinlocky:** Typ zámku, kde vlákno místo uspání aktivně čeká (spínuje) v cyklu, dokud se zámek neuvolní. Vhodné pro velmi krátké kritické sekce, kde je očekávaná doba čekání kratší než režie přepnutí kontextu.

## Klasické problémy synchronizace:

- **Deadlock (Uváznutí):** Křížové zablokování. Nastává, když dvě nebo více vláken/procesů čekají navzájem na prostředky, které drží to druhé.
    - **Podmínky pro deadlock (Coffmanovy podmínky):**
      1. **Vzájemné vyloučení:** Alespoň jeden prostředek musí být držen v nevyměnitelné (exkluzivní) podobě.
      2. **Držení a čekání:** Proces drží alespoň jeden prostředek a čeká na další, který drží jiný proces.
      3. **Žádné preempce:** Prostředek nemůže být odebrán procesu, který ho drží, dokud ho sám neuvolní.
      4. **Kruhové čekání:** Existuje kruhový řetězec procesů, kde každý proces čeká na prostředek držený dalším procesem v řetězci.
    - **Příklad:** Vlákno A drží Mutex 1 a čeká na uvolnění Mutexu 2. Vlákno B drží Mutex 2 a čeká na uvolnění Mutexu 1. Obě vlákna budou navždy spát.
  - **Starvation (Hladovění):** Situace, kdy je vlákno kvůli špatně nastaveným prioritám plánovače nebo nevhodnému algoritmu přidělování prostředků neustále předbíháno jinými vlákny a nikdy se nedostane k procesoru nebo k požadovanému zdroji.
  - **Problém spících filozofů (Dining Philosophers Problem):** Klasický problém ilustrující deadlock a starvation, kde pět filozofů sedí u kulatého stolu, střídavě jí a přemýšlí. Mezi každými dvěma filozofy je jedna hůlka. Každý filozof potřebuje dvě hůlky (jednu zleva, jednu zprava), aby mohl jíst. Pokud všichni filozofové uchopí svou levou hůlku současně, dojde k deadlocku, protože každý čeká na pravou hůlku, kterou drží jeho soused.
-

## Praktické použití

- **OS API:**
    - Práce se soubory a adresáři.
    - Správa procesů a vláken.
    - Síťová komunikace.
    - Přístup k hardwarovým zařízením.
    - Získávání systémových informací (čas, ID procesů).
  - **Zpracování zpráv:**
    - Grafická uživatelská rozhraní (GUI).
    - Asynchronní I/O operace (např. v webových serverech).
    - Meziprocesová komunikace v distribuovaných systémech.
    - Architektury mikroslužeb.
  - **Vlákna a synchronizace:**
    - Paralelní výpočty (např. vědecké simulace, zpracování obrazu/video).
    - Responzivní uživatelská rozhraní (GUI, které nezamrzá).
    - Webové servery (zpracování více požadavků současně).
    - Databázové systémy (souběžný přístup k datům).
    - Optimalizace výkonu na víceprocesorových systémech.
- 

## Nejčastější chyby u zkoušky

- **Zaměňování procesu a vlákna:** Nejasné pochopení rozdílů v izolaci paměti a sdílení zdrojů.
- **Ignorování režie systémových volání:** Představa, že systémová volání jsou stejně rychlá jako volání běžných funkcí.
- **Neznalost mechanismu systémového volání:** Nechápaní přepnutí režimu jádra/uživatele.
- **Nepochopení potřeby synchronizace:** Předpoklad, že kód bude fungovat správně i bez ochrany sdílených zdrojů.
- **Příliš široké kritické sekce:** Zamykání příliš velkých částí kódu, což snižuje paralelizmus.

- **Neřešení chyb systémových volání:** Ignorování návratových hodnot a chybových kódů.
  - **Neznalost podmínek pro deadlock:** Neumět vysvětlit, proč deadlock nastává.
  - **Záměna mutexu a semaforu:** Nejasné pochopení, kdy použít který primitiv.
  - **Blokování hlavního GUI vlákna:** Provádění dlouhých operací v událostní smyčce, což vede k zamrzání aplikace.
- 

## Typické zkuškové otázky

### 1. Co je systémové volání (syscall) a proč je potřeba?

- **Odpověď:** Systémové volání je mechanismus, kterým program v uživatelském režimu (user space) požádá jádro operačního systému (kernel space) o provedení privilegované operace nebo služby (např. přístup k hardwaru, správa paměti, I/O operace). Je potřeba pro zajištění bezpečnosti a stability systému, protože aplikace nemají přímý přístup k hardwaru a privilegovaným zdrojům. Jádro tak funguje jako prostředník, který ověřuje oprávnění a spravuje systémové zdroje.

### 2. Vysvětlete rozdíl mezi procesem a vláknem.

- **Odpověď:** **Proces** je spuštěná instance programu s vlastním izolovaným paměťovým prostorem, vlastními registry, otevřenými soubory a dalším kontextem. Procesy jsou od sebe striktně oddělené. **Vláknem** je nejmenší jednotka vykonávání kódu v rámci procesu. Vlákna jednoho procesu sdílejí stejný paměťový prostor (haldy, globální proměnné, kód), ale každé má svůj vlastní zásobník (stack) a ukazatel instrukcí (Instruction Pointer). Přepínání mezi vlákny je rychlejší než mezi procesy.

### 3. Co je kritická sekce a jaký je její vztah k race condition?

- **Odpověď:** **Kritická sekce** je část kódu, která přistupuje ke sdílenému zdroji (např. globální proměnné, souboru) a u které je

nutné zajistit, aby ji v jeden okamžik vykonávalo maximálně jedno vlákno nebo proces. **Race condition (souběh)** nastává, když více vláken nebo procesů přistupuje ke sdílenému zdroji (a alespoň jedno z nich provádí zápis) bez řádné synchronizace. Výsledek operace se pak stává závislým na náhodném pořadí vykonávání instrukcí, což vede k nepředvídatelnému a chybnému chování. Kritická sekce je tedy kód, který musí být chráněn, aby se zabránilo race condition.

4. **Popište funkci mutexu a semaforu a uveďte, kdy byste který použili.**

○ **Odpověď:**

- **Mutex** (Mutual Exclusion) je synchronizační primitivum, které funguje jako binární zámek. Vlákno, které chce vstoupit do kritické sekce, se pokusí mutex zamknout. Pokud je volný, uzamkne ho a vstoupí. Pokud je zamčený, vlákno se zablokuje a čeká. Po dokončení kritické sekce vlákno mutex odemkne. Používá se, když potřebujeme zajistit, aby ke sdílenému zdroji přistupovalo v jeden okamžik **maximálně jedno** vlákno (vzájemné vyloučení).
- **Semafor** je zobecněný mutex, který obsahuje čítač. Udává, kolik vláken může současně přistupovat ke zdroji. Operace `wait()` dekrementuje čítač (a blokuje, pokud je záporný), operace `signal()` inkrementuje čítač (a probudí čekající vlákno, pokud existuje). **Binární semafor** (s čítačem 0 nebo 1) se chová jako mutex. **Čítací semafor** se používá, když chceme omezit přístup ke zdroji na **konkrétní počet** vláken současně (např. fond připojení k databázi s omezenou kapacitou).

5. **Vysvětlete pojmy deadlock a starvation.**

○ **Odpověď:**

- **Deadlock (uváznutí)** je situace, kdy dvě nebo více vláken/procesů čekají navzájem na prostředky, které drží to

druhé, a žádné z nich nemůže pokračovat. Typickým příkladem je kruhové čekání na zámky.

- **Starvation (hladovění)** je situace, kdy vlákno/proces není schopno získat přístup k požadovanému zdroji nebo procesoru po delší dobu, i když je zdroj k dispozici. To může být způsobeno nevhodným plánováním priorit nebo neférovým přidělováním zdrojů, kdy jsou jiná vlákna neustále upřednostňována.

## 6. Jak funguje zpracování zpráv v GUI aplikacích?

- **Odpověď:** V GUI aplikacích je zpracování zpráv (event-driven programming) klíčové pro interakci s uživatelem a systémem. Uživatelské akce (kliknutí myši, stisknutí klávesy), systémové události (změna velikosti okna, časovače) a interní události aplikace jsou generovány jako zprávy. Tyto zprávy jsou vkládány do **fronty zpráv**. Hlavní vlákno aplikace běží v nekonečné **událostní smyčce**, která neustále vybírá zprávy z fronty a předává je příslušným **obslužným rutinám (callback funkcím)**, které provedou odpovídající akci. Tím je zajištěna asynchronní a responzivní interakce s uživatelem.

---

## Shrnutí kapitoly

Operační systém poskytuje aplikacím klíčové služby prostřednictvím **API a systémových volání**, které zajišťují bezpečnost, abstrakci a efektivní správu zdrojů. Pro moderní, responzivní a výkonné aplikace jsou nezbytné pokročilé programátorské techniky. **Zpracování zpráv** je základem pro event-driven architektury a GUI, kde **událostní smyčka** asynchronně reaguje na podněty. **Programování vláken (multithreading)** umožňuje využít paralelismus moderních procesorů a zlepšit odezvu aplikací, avšak přináší s sebou výzvy v podobě **race conditions**. K jejich řešení a zajištění konzistence sdílených dat slouží **synchronizační primitiva** jako **mutexy** (pro exkluzivní přístup), **semaforey** (pro omezený souběžný přístup) a **podmínkové proměnné** (pro



signalizaci). Nesprávné použití těchto technik může vést k závažným problémům, jako jsou **deadlock** a **starvation**.

---

Zde je dokonalý studijní materiál k okruhu "18. Operační systém, vysvětlení pojmu, typy, poskytované funkce", doplněný a naformátovaný dle vašich přísných pravidel.

---

## 18. Operační systém, vysvětlení pojmu, typy, poskytované funkce

---

### A. Vysvětlení pojmu Operační systém (OS)

Operační systém (OS) je **komplexní balík programového vybavení**, který funguje jako **prostředník mezi hardwarovými prostředky počítače a aplikačním softwarem** (uživatelskými programy). Jeho primární role lze shrnout do dvou hlavních funkcí:

- **Správce prostředků (Resource Manager):** OS efektivně přiděluje a spravuje hardwarové prostředky (CPU, paměť, I/O zařízení) mezi běžící programy, aby zajistil jejich spravedlivé a efektivní využití.
- **Rozšířený stroj (Extended Machine):** OS poskytuje aplikacím zjednodušenou a abstraktní reprezentaci hardwaru, čímž skrývá složité nízkourovňové detaily ovládání zařízení a usnadňuje programování.

**Základním jádrem operačního systému je Kernel (jádro).** Kernel se zavádí do paměti RAM při startu počítače (tzv. **Bootování**) a běží v tzv. **Privilegovaném režimu (Kernel Mode / Supervisor Mode)**. Tento režim mu dává neomezený přístup ke všem instrukcím procesoru a hardwaru. Běžné uživatelské aplikace běží v **Neprivilegovaném režimu (User Mode)** a k hardwaru smí přistupovat pouze nepřímo skrze **systémová volání (Syscalls)**, která jsou obsluhována jádrem. Tím je zajištěna bezpečnost a stabilita systému, protože chybná aplikace nemůže přímo poškodit hardware nebo data jiných aplikací.

## B. Poskytované funkce (Služby OS)

Operační systém plní řadu klíčových funkcí, které jsou nezbytné pro chod počítače a aplikací:

- **Správa procesů a vláken (Process Management):**
  - **Vytváření, plánování, synchronizace a ukončování procesů.**  
Proces je běžící instance programu s vlastním adresním prostorem a systémovými prostředky.
  - **Plánovač (Scheduler) OS** přepíná procesy na jádrech CPU tak rychle, že uživatel má pocit, že všechny programy běží současně (tzv. **multitasking**).
  - Zajišťuje **synchronizaci** mezi procesy a vlákny (např. pomocí mutexů, semaforů) a řeší problémy jako **deadlock** nebo **race condition**.
  - Spravuje **životní cyklus procesu** (nový, připravený, běžící, čekající, ukončený).
- **Správa paměti (Memory Management):**
  - **Přidělování a uvolňování operační paměti RAM** pro běžící procesy a data.
  - OS zajišťuje **izolaci paměti** – žádný proces nesmí číst nebo přepisovat paměť jiného procesu, pokud to není explicitně dovoleno.
  - Využívá **virtuální paměť**, která umožňuje aplikacím pracovat s větším adresním prostorem, než je fyzicky dostupná RAM. Toho je dosaženo pomocí **stránkování (paging)**, kdy se části paměti přesouvají mezi RAM a diskem (tzv. **swapping**).
- **Správa souborů (File System):**
  - **Abstrakce dat na úložných médiích** (SSD, HDD). OS organizuje surové bajty na disku do logické struktury **adresářů a souborů**.
  - Řídí **přístupová práva** k souborům a adresářům (kdo může číst, zapisovat, spouštět).

- Poskytuje jednotné rozhraní pro práci se soubory, bez ohledu na typ úložného zařízení.
- **Správa periferních zařízení (I/O Management):**
  - **Ovládání hardwaru** (grafické karty, disky, síťové adaptéry, tiskárny) pomocí specializovaných programů zvaných **ovladače (drivers)**.
  - Poskytuje aplikacím **jednotné rozhraní** pro čtení a zápis dat z/do zařízení.
  - Zajišťuje efektivní a bezpečný přístup k I/O zařízením.
- **Bezpečnost a ochrana:**
  - **Autentizace uživatelů** (např. přihlašování pomocí jména a hesla).
  - **Autorizace** (řízení přístupových práv k souborům a systémovým prostředkům na základě identity uživatele).
  - Ochrana před škodlivým kódem a neoprávněným přístupem.
- **Uživatelské rozhraní (User Interface):**
  - Umožňuje člověku OS ovládat a interagovat s ním.
  - **Textové rozhraní (CLI – Command Line Interface):** Např. Bash, PowerShell, Command Prompt. Uživatel zadává příkazy textově.
  - **Grafické rozhraní (GUI – Graphical User Interface):** Např. Windows Desktop, GNOME, KDE, macOS Aqua. Uživatel interaguje s vizuálními prvky (okna, ikony, menu).

## C. Typy operačních systémů

Operační systémy můžeme klasifikovat podle několika kritérií:

### 1. Podle architektury jádra (Kernelu)

- **Monolitické jádro (Monolithic Kernel):**

- **Princip:** Celé jádro (správa paměti, procesů, souborové systémy, síťové protokoly a většina ovladačů zařízení) běží jako jeden obří program v privilegovaném režimu.
- **Výhoda:** Extrémní rychlost (volání funkcí uvnitř jádra má minimální režii, protože vše je ve stejném adresním prostoru).
- **Nevýhoda:** Nízká stabilita a modularita. Pokud selže nebo spadne jeden ovladač (např. grafické karty), spadne celé jádro a s ním celý počítač (např. Blue Screen of Death ve starších Windows). Obtížnější ladění a vývoj.
- **Příklady:** Linux, tradiční Unix (např. Solaris), starší MS-DOS.

- **Mikrojádru (Microkernel):**

- **Princip:** V privilegovaném režimu běží jen naprosté minimum funkcí (správa adresního prostoru, plánování vláken a meziprocsová komunikace – **IPC**). Všechno ostatní (souborové systémy, ovladače zařízení, síťové protokoly) běží jako běžné procesy v uživatelském prostoru.
- **Výhoda:** Vysoká stabilita a modularita. Pokud spadne ovladač disku, OS ho jednoduše restartuje jako běžný program, zbytek systému běží dál. Snadnější vývoj a údržba.
- **Nevýhoda:** Pomalejší běh kvůli neustálému **přepínání kontextu (Context Switch)** a režii při posílání zpráv přes IPC mezi komponentami jádra.
- **Příklady:** QNX (používá se v autech a kritických systémech), MINIX, Mach (základ pro macOS/iOS).

- **Hybridní jádro (Hybrid Kernel):**

- **Princip:** Kombinace obou přístupů. Architektonicky se podobá mikrojádru, ale z výkonnostních důvodů jsou některé klíčové komponenty (např. grafický subsystém, síťový stack) přesunuty do privilegovaného režimu, aby se snížila rezie IPC.
- **Výhoda:** Snaží se spojit stabilitu mikrojádru s výkonem monolitického jádra.
- **Nevýhoda:** Stále složitější než mikrojádru, může mít některé problémy se stabilitou monolitických jader.

- **Příklady:** Windows NT (jádro moderních Windows 10/11), macOS/iOS (jádro XNU).

## 2. Podle účelu použití a chování

- **OS pro osobní počítače a servery (General-Purpose OS):**

- **Charakteristika:** Zaměřené na vysoký celkový výkon (throughput), uživatelský komfort, širokou kompatibilitu s hardwarem a softwarem. Podporují **preemptivní multitasking** a virtuální paměť.
- **Využití:** Desktopové počítače, notebooky, servery, datová centra.
- **Příklady:** Microsoft Windows, Linux distribuce (Ubuntu, Fedora), macOS.

- **Operační systémy reálného času (RTOS - Real-Time Operating Systems):**

- **Charakteristika:** U těchto systémů není nejdůležitější průměrná rychlost, ale **determinismus**. OS musí garantovat, že určitá úloha (např. reakce na senzor, řízení motoru, vystřelení airbagu) se provede v přesně definovaném časovém limitu. Rozlišujeme **Hard Real-Time** (selhání termínu je katastrofa) a **Soft Real-Time** (selhání termínu je nežádoucí, ale ne kritické).
- **Využití:** Řízení průmyslových robotů, automobilový průmysl (ABS, airbagy), avionika, lékařské přístroje, embedded zařízení s kritickými časovými požadavky.
- **Příklady:** QNX, FreeRTOS, VxWorks.

- **Síťové operační systémy (Network OS):**

- **Charakteristika:** Navržené primárně pro správu síťových prostředků, sdílení souborů a tiskáren, a řízení přístupu více uživatelů napříč sítí. Každý počítač v síti má svůj vlastní OS a je si vědom existence ostatních.
- **Využití:** Firemní sítě, servery pro sdílení zdrojů.
- **Příklady:** Windows Server, některé Linux distribuce.

- **Distribuované operační systémy (Distributed OS):**

- **Charakteristika:** Cílem je, aby kolekce nezávislých počítačů (uzlů) fungovala jako jeden koherentní systém. Uživatelé vnímají celou síť jako jeden velký počítač. Zajišťují transparentnost (uživatel neví, kde se data nebo procesy fyzicky nacházejí).
- **Využití:** Velké distribuované systémy, cloud computing.
- **Příklady:** Amoeba, Chorus (spíše akademické a výzkumné systémy, moderní distribuované systémy jsou často postaveny na middleware nad standardními OS).

- **Embedded operační systémy (Embedded OS):**

- **Charakteristika:** Navrženy pro specifické účely v zařízeních s omezenými zdroji (paměť, procesor, spotřeba energie). Často jsou optimalizovány pro rychlý start a spolehlivost.
- **Využití:** Chytré televize, pračky, mikrovlnné trouby, průmyslové řídicí jednotky, IoT zařízení.
- **Příklady:** FreeRTOS, uCOS, některé verze Linuxu (Embedded Linux).

- **Mobilní operační systémy (Mobile OS):**

- **Charakteristika:** Optimalizovány pro dotykové rozhraní, správu napájení, konektivitu (Wi-Fi, mobilní sítě), a ekosystém aplikací.
- **Využití:** Chytré telefony, tablety.
- **Příklady:** Android, iOS.

---

## Typické zkouškové otázky

### 1. Definujte operační systém, vysvětlete jeho dvě hlavní role a rozdíl mezi uživatelským a jádrovým režimem.

- **Odpověď:** Operační systém je komplexní programové vybavení, které funguje jako prostředník mezi hardwarem a aplikačním softwarem. Jeho dvě hlavní role jsou:
  - **Správce prostředků:** Efektivně přiděluje a spravuje hardwarové prostředky (CPU, paměť, I/O) mezi běžící programy.

- **Rozšířený stroj:** Poskytuje aplikacím zjednodušenou a abstraktní reprezentaci hardwaru, skrývající nízkoúrovňové detaily.
- **Uživatelský režim (User Mode)** je režim, ve kterém běží běžné aplikace s omezenými oprávněními, nemohou přímo přistupovat k hardwaru. **Jádrový režim (Kernel Mode)** je privilegovaný režim, ve kterém běží jádro OS s plným přístupem ke všem instrukcím procesoru a hardwaru. Aplikace v uživatelském režimu musí pro přístup k hardwaru využít systémová volání, která jsou obsluhována jádrem v jádrovém režimu.

**2. Popište alespoň čtyři hlavní funkce operačního systému a u každé uveďte příklad, jak ji OS realizuje.**

○ **Odpověď:**

- **Správa procesů a vláken:** OS vytváří, plánuje a ukončuje procesy. Příkladem je **plánovač (scheduler)**, který přepíná mezi běžícími programy na CPU (multitasking), aby se zdálo, že běží současně.
- **Správa paměti:** OS přiděluje a uvolňuje RAM pro programy. Příkladem je **virtuální paměť**, která umožňuje programům používat více paměti, než je fyzicky k dispozici, a to přesouváním dat mezi RAM a diskem (stránkování).
- **Správa souborů:** OS organizuje data na disku do logické struktury souborů a adresářů. Příkladem je **souborový systém (např. NTFS, ext4)**, který spravuje ukládání, načítání a přístupová práva k datům.
- **Správa periferních zařízení:** OS ovládá hardware připojený k počítači. Příkladem jsou **ovladače (drivers)**, které umožňují OS komunikovat s konkrétními zařízeními, jako je tiskárna nebo grafická karta, a poskytují aplikacím jednotné rozhraní.

**3. Porovnejte monolitické a mikrojádrové architektury operačních systémů, včetně jejich výhod a nevýhod a uveďte příklady.**



○ **Odpověď:**

- **Monolitické jádro:** Celé jádro (včetně ovladačů, souborových systémů atd.) běží jako jeden celek v privilegovaném režimu.
  - **Výhody:** Vysoký výkon díky minimální režii interních volání.
  - **Nevýhody:** Nízká stabilita (selhání jedné komponenty může shodit celý systém), obtížnější vývoj a ladění.
  - **Příklady:** Linux, tradiční Unix.
- **Mikrojádru:** V privilegovaném režimu běží jen minimum funkcí (správa paměti, plánování, IPC). Ostatní služby běží jako uživatelské procesy.
  - **Výhody:** Vysoká stabilita a modularita (selhání služby neshodí celé jádro), snazší vývoj.
  - **Nevýhody:** Nižší výkon kvůli režii meziprocesové komunikace (IPC) a častému přepínání kontextu.
  - **Příklady:** QNX, MINIX.

4. **Vysvětlete pojem virtuální paměť a proč je důležitá pro moderní operační systémy.**

- **Odpověď:** Virtuální paměť je technika správy paměti, která umožňuje programu používat větší adresní prostor, než je fyzicky dostupná operační paměť (RAM). OS toho dosahuje mapováním virtuálních adres na fyzické adresy a přesouváním neaktivních částí paměti (stránek) na disk (tzv. stránkování nebo swapping).
- **Důležitost:**
- Umožňuje spouštět více programů najednou, i když jejich celková paměťová náročnost přesahuje kapacitu RAM.
  - Poskytuje **izolaci paměti** mezi procesy, čímž zvyšuje bezpečnost a stabilitu systému.
  - Zjednodušuje programování, protože programátoři nemusí řešit fyzické adresy paměti.

5. **Co je to Real-Time Operating System (RTOS) a v jakých oblastech se typicky používá?**

- **Odpověď:** RTOS je operační systém, který garantuje, že určité úlohy budou dokončeny v přesně definovaném časovém limitu (determinizmus). Na rozdíl od běžných OS není primárním cílem průměrná rychlost, ale předvídatelnost a spolehlivost časování. Rozlišujeme **Hard Real-Time** (absolutní dodržení termínu) a **Soft Real-Time** (preferované dodržení termínu).
  - **Typické oblasti použití:**
    - **Průmyslová automatizace:** Řízení robotů, výrobních linek.
    - **Automobilový průmysl:** Řídicí jednotky motoru, ABS, airbagy.
    - **Avionika:** Letové řídicí systémy.
    - **Lékařské přístroje:** Monitory životních funkcí, infuzní pumpy.
    - **Embedded systémy:** Zařízení s kritickými časovými požadavky, kde je nutná okamžitá reakce na události.
-

Níže je připravený studijní materiál k okruhu 19, doplněný a naformátovaný dle vašich přísných pravidel.

---

## 19. Správa procesů v operačním systému, vztah programu a procesu, životní cyklus procesu

---

### Klíčové pojmy

- proces
- stav procesu
- PCB (Process Control Block)
- plánování
- kontext
- fork
- exec
- zombie proces

### A. Vztah programu a procesu

Ačkoliv se tyto dva pojmy v běžné řeči zaměňují, z pohledu informatiky je mezi nimi zásadní rozdíl:

- **Program (Pasivní entita):**
  - Je to statický soubor uložený na pevném disku nebo SSD (např. kompilovaný binární soubor `.exe` ve Windows nebo ELF v Linuxu).
  - Obsahuje instrukce pro procesor a statická data, která "spí", dokud je uživatel nespustí.

- **Proces (Aktivní entita):**

- Je to spuštěná (vykonávaná) instance programu v operační paměti RAM.
- Má přidělené systémové prostředky (adresní prostor v paměti, procesorový čas, deskriptory otevřených souborů, síťové sockety).

**Příklad:** Pokud máš na disku prohlížeč Chrome, je to program. Pokud ho otevřeš ve třech nezávislých oknech, v paměti RAM ti vzniknou tři samostatné procesy, které sdílejí stejný kód programu, ale každý má svá vlastní uživatelská data a paměť.

## B. Struktura procesu v paměti

Když operační systém (OS) vytváří proces, vyhradí mu adresní prostor, který se typicky dělí do čtyř základních segmentů:

- **Textový segment (Code):**

- Obsahuje samotné binární strojové instrukce procesoru, které se čtou a vykonávají.
- Tento segment je obvykle určen pouze pro čtení (Read-Only), aby se zabránilo jeho nechtěné modifikaci.

- **Datový segment (Data):**

- Obsahuje globální a statické proměnné.
- Dělí se na inicializovaná data (proměnné s předdefinovanou hodnotou) a neinicializovaná data (proměnné, které jsou inicializovány na nulu nebo ponechány neinicializované).

- **Halda (Heap):**

- Prostor pro dynamicky alokovanou paměť za běhu programu (pomocí funkcí jako `malloc` v C nebo operátoru `new` v objektově orientovaném programování).
- Roste směrem nahoru k vyšším paměťovým adresám.

- **Zásobník (Stack):**

- Slouží k ukládání lokálních proměnných, parametrů funkcí a návratových adres volaných funkcí.
- Roste směrem dolů k nižším paměťovým adresám.

## C. Řízení procesu: PCB (Process Control Block)

Operační systém si o každém běžícím procesu musí udržovat detailní přehled. K tomu používá speciální datovou strukturu v jádře zvanou **PCB (Process Control Block)**. PCB obsahuje klíčové informace o procesu:

- **PID (Process ID):** Unikátní identifikační číslo procesu v systému.
- **Stav procesu:** Aktuální fáze v životním cyklu (např. Nový, Připravený, Běžící, Čekající, Ukončený).
- **Kontext procesoru:** Uložené hodnoty všech registrů CPU (včetně Instruction Pointeru a Stack Pointeru) pro případ, že plánovač proces na chvíli pozastaví, aby dal prostor jinému procesu.
- **Informace o správě paměti:** Ukazatele na tabulky stránek virtuální paměti, které definují adresní prostor procesu.
- **Vlastník a oprávnění:** Informace o tom, který uživatel proces spustil a jaká oprávnění má proces v systému.
- **Seznam otevřených souborů a prostředků:** Deskriptory všech souborů, síťových socketů a dalších systémových prostředků, které proces aktuálně používá.

## D. Životní cyklus a stavy procesu

Během své existence prochází proces několika stavy. Typický stavový model obsahuje tyto základní fáze:

- **Nový (New / Created):**
  - Proces se právě vytváří.
  - OS alokuje paměť, zakládá PCB, ale proces ještě není připraven k samotnému spuštění.

- **Připravený (Ready):**

- Proces má přidělenou veškerou potřebnou paměť i prostředky a čeká pouze na to, až mu plánovač OS (Scheduler) přidělí procesorový čas (CPU).
- V tomto stavu může v paměti čekat fronta mnoha procesů.

- **Běžící (Running):**

- Plánovač vybral proces z fronty připravených, nahrál jeho kontext do registrů a procesor aktuálně vykonává jeho instrukce.
- Na jednom jádru CPU může být v daný okamžik pouze jeden běžící proces.

- **Čekající / Blokováný (Waiting / Blocked):**

- Proces nemůže běžet, protože čeká na dokončení nějaké vnější události (např. na načtení dat z disku, stisk klávesy uživatelem, příchod síťového paketu nebo uvolnění synchronizačního mutexu).
- V tomto stavu nespotřebovává CPU.
- Jakmile událost nastane, OS ho přesune zpět do stavu Připravený.

- **Ukončený (Terminated / Zombie):**

- Proces dokončil svou práci (zavolal `exit` nebo `return` z `main`), nebo byl násilně ukončen systémem (např. při chybě paměti).
- Jeho prostředky jsou uvolněny, ale záznam v tabulce procesů (PCB) zůstává aktivní, dokud si jeho rodičovský proces nepřechte návratový kód. Tento stav se někdy nazývá "zombie".

### **Přechody mezi stavy:**

- **Ready → Running:** Zásah plánovače OS (Dispatch), který vybere proces z fronty připravených a přidělí mu CPU.
- **Running → Ready:** Vypršel časový kvant, který měl proces přidělený (u preemptivního multitaskingu), nebo byl předběhnut procesem s

vyšší prioritou.

- **Running** → **Waiting**: Proces zažádal o pomalou I/O operaci (např. čtení z disku) nebo se zablokoval na synchronizačním primitivu (např. mutexu).
- **Waiting** → **Ready**: I/O operace byla dokončena (zpravidla signalizováno hardwarovým přerušením) nebo synchronizační primitivum bylo uvolněno.
- **Running** → **Terminated**: Proces dokončil svou práci nebo byl ukončen chybou/systémem.

## E. Přepínání kontextu (Context Switch)

Přepínání kontextu je mechanismus, který umožňuje multitasking na jednom jádru CPU. Když plánovač rozhodne, že běžící proces  $A$  musí uvolnit místo procesu  $B$ :

1. OS pozastaví vykonávání procesu  $A$ .
2. Uloží aktuální stav všech registrů CPU (včetně ukazatele instrukcí) do PCB procesu  $A$ .
3. Načte uložený stav registrů z PCB procesu  $B$  zpět do procesoru.
4. Změní mapování virtuální paměti na adresní prostor procesu  $B$ .
5. Procesor začne vykonávat instrukce procesu  $B$  přesně tam, kde minule přestal.

⚠ Přepínání kontextu představuje čistou výpočetní režii. Během přepínání procesor nevykonává žádný užitečný kód aplikace, pouze manipuluje s pamětí jádra. Proto se operační systémy snaží navrhovat plánovače tak, aby k přepínání nedocházelo zbytečně často, a minimalizovaly tak režii.

## Praktické použití

- **Multitasking**: Umožňuje spouštět více programů "současně" na jednom CPU.
- **Izolace aplikací**: Každý proces má svůj vlastní adresní prostor, což zabraňuje jednomu programu poškodit data jiného.

- **Správa prostředků:** OS může efektivně přidělovat a odebírat CPU čas, paměť a další prostředky jednotlivým procesům.
- **Stabilita systému:** Izolace procesů a jejich řízený životní cyklus zvyšuje celkovou stabilitu operačního systému.

## Nejčastější chyby u zkoušky

- **Zaměňování programu a procesu:**
  - Program je pasivní soubor na disku, proces je jeho aktivní instance v paměti.
- **Neuvědomění si režie přepínání kontextu:**
  - Přepínání kontextu není zdarma, spotřebovává CPU čas na ukládání a načítání stavu procesoru.
- **Neznalost stavu waiting při I/O:**
  - Proces ve stavu *Waiting* aktivně nekonzumuje CPU čas, ale čeká na dokončení I/O operace nebo na jinou událost.

## Typické zkouškové otázky

### 1. Popište životní cyklus procesu.

- **Odpověď:** Životní cyklus procesu zahrnuje stavy Nový (proces se vytváří), Připravený (čeká na CPU), Běžící (vykonává instrukce na CPU), Čekající/Blokovaný (čeká na I/O nebo událost) a Ukončený (dokončil práci nebo byl ukončen). Proces se přesouvá mezi těmito stavy na základě akcí plánovače, I/O operací nebo dokončení své činnosti.

### 2. Co obsahuje PCB?

- **Odpověď:** PCB (Process Control Block) je datová struktura v jádře OS, která obsahuje veškeré informace o procesu. Mezi klíčové položky patří PID (Process ID), aktuální stav procesu,



kontext procesoru (hodnoty registrů CPU), informace o správě paměti (např. ukazatele na tabulky stránek), vlastník a oprávnění procesu a seznam otevřených souborů a dalších systémových prostředků.

### 3. Jak se liší proces a vlákno?

- **Odpověď:** Proces je samostatná instance programu s vlastním adresním prostorem, PCB a sadou systémových prostředků. Vlákno je lehčí jednotka vykonávání v rámci jednoho procesu. Vlákna v rámci stejného procesu sdílejí adresní prostor, otevřené soubory a další prostředky procesu, ale mají svůj vlastní zásobník, registry a program counter. Procesy poskytují izolaci, zatímco vlákna umožňují souběžné vykonávání úloh v rámci jedné aplikace s menší režii při přepínání.

## Shrnutí kapitoly

Proces je základní jednotka izolace a správy v operačním systému. Jeho životní cyklus řídí plánovač podle dostupnosti CPU a I/O operací. Pochopení vztahu mezi programem a procesem, struktury procesu v paměti a role PCB je klíčové pro porozumění fungování operačních systémů a multitaskingu. Přepínání kontextu je nezbytné pro souběžné vykonávání více procesů, ale představuje režii, kterou se OS snaží minimalizovat.

---